

NexusAI

Asistente académico con IA integrado en Moodle

Santiago Tricherri

Delfina Salinas

2026

Abstract

TODO — Resumen ejecutivo del proyecto NexusAI: asistente académico basado en IA con RAG real sobre el material de cada curso, integrado como plugin de Moodle. Este documento presenta la entrega final del proyecto correspondiente a la asignatura Administración de Proyectos de Software.

Contents

1	Resumen ejecutivo	10
1.1	Problema	10
1.2	Solución propuesta	10
1.3	Diferenciadores técnicos	11
1.4	Estado del proyecto al cierre del MVP	11
1.5	Próximos pasos post-MVP	11
2	Introducción y contexto	12
2.1	Contexto	12
2.2	Motivación	12
2.3	Objetivos del proyecto	13
2.3.1	Objetivo general	13
2.3.2	Objetivos específicos	13
2.4	Alcance de este documento	13
2.5	Equipo	13
2.6	Universidad e institución	14
3	Alcance del proyecto	15
3.1	Producto esperado	15
3.2	In scope — funcionalidades del MVP	15
3.2.1	Plugin Moodle	15
3.2.2	Backend FastAPI	15
3.2.3	Interfaz de chat	15
3.2.4	Gestión de material por parte del docente	16
3.2.5	Funcionalidades complementarias del MVP	16
3.2.6	Calidad y operación	16
3.2.7	Validación	16
3.3	Out of scope — explícitamente descartado del MVP	16
3.4	Usuarios objetivo	17
3.4.1	Rol primario: alumno	17
3.4.2	Rol secundario: docente	17
3.4.3	Rol terciario: administrador de Moodle	17
3.5	Restricciones del proyecto	17
4	Análisis de requerimientos	19
4.1	Convenciones	19
4.2	Requerimientos funcionales	19

4.2.1	Asistente conversacional RAG	19
4.2.2	Buscador semántico	20
4.2.3	Quiz generator	21
4.2.4	Gestión de material (rol docente)	21
4.2.5	Feedback loop al docente (detección de gaps)	22
4.3	Requerimientos no funcionales	23
4.3.1	Seguridad	23
4.3.2	Performance	23
4.3.3	Compatibilidad	24
4.3.4	Privacidad y soberanía de datos	24
4.3.5	Mantenibilidad	24
4.3.6	Distribución	25
5	Historias de usuario y criterios de aceptación	26
5.1	Convenciones	26
5.2	Resumen agregado del backlog	26
5.3	Épica 1 — Asistente conversacional con RAG	27
5.3.1	HU-001 — Chat básico del alumno	27
5.3.2	HU-002 — Citas trazables a la fuente	27
5.3.3	HU-003 — Streaming token-por-token	27
5.4	Épica 2 — Gestión de material por parte del docente	28
5.4.1	HU-010 — Upload de PDFs por el docente	28
5.4.2	HU-011 — Detección de duplicados	28
5.5	Épica 3 — Buscador semántico (Feature A)	28
5.5.1	HU-020 — Buscar fragmentos por similaridad	28
5.6	Épica 4 — Quiz generator (Feature F)	29
5.6.1	HU-030 — Generar quiz desde el material	29
5.6.2	HU-031 — Tema sin material disponible	29
5.7	Épica 5 — Historial de conversaciones (Feature E)	29
5.7.1	HU-040 — Lista de sesiones previas	29
5.8	Épica 6 — Multi-curso (Feature B)	30
5.8.1	HU-050 — Activar modo multi-curso	30
5.9	Épica 7 — Detección de gaps del docente (Feature G)	30
5.9.1	HU-060 — Ver gaps detectados	30
6	Work Breakdown Structure (WBS)	31
6.1	Descomposición jerárquica del proyecto	31
6.2	Cargas de trabajo por entregable	33
7	Cronograma del proyecto	34
7.1	Fases del proyecto	34
7.2	Diagrama Gantt	35
7.3	Detalle por sprint	35
7.3.1	Sprint 0 — Setup e Investigación (9 - 22 Abr 2026)	35
7.3.2	Sprint 1 — Core chat (23 Abr - 6 May 2026)	36
7.3.3	Sprint 2 — RAG y carga de material (7 - 20 May 2026)	36
7.3.4	Sprint 3 — Calidad y métricas (21 - 27 May 2026)	36
7.3.5	Sprint 4 — MVP completo (28 May - 1 Jun 2026)	36

7.3.6	Documentación MVP (2 - 15 Jun 2026)	37
7.4	Velocidad real vs estimada	37
8	Estimaciones	38
8.1	Metodología	38
8.2	Story points por épica	38
8.3	Story points por sprint	39
8.4	Conversión SP $\square$ horas-persona	39
8.5	Estimaciones por integrante	39
8.6	Carry-over y deuda técnica	40
8.7	Comparación con estimación inicial	40
9	Costos	42
9.1	Metodología	42
9.2	Costos de infraestructura	42
9.2.1	Escenario MVP (estado actual, junio 2026)	42
9.2.2	Escenario producción (post-MVP, 500 alumnos)	42
9.3	Costos de servicios de IA	43
9.3.1	MVP con Gemini (tier gratuito)	43
9.3.2	Producción con OpenAI	43
9.4	Costos de personal (valoración teórica)	44
9.5	Costo total proyectado	44
9.5.1	Para la tesis (estado actual)	44
9.5.2	Para producción comercial (1 año, 500 alumnos)	44
9.6	Análisis costo-beneficio	45
10	Análisis de riesgos	46
10.1	Metodología	46
10.2	Matriz de riesgos	46
10.3	Plan detallado de los riesgos críticos (severidad ≥ 6)	49
10.3.1	R-01 — Cambio en políticas de Gemini API	49
10.3.2	R-04 — Rotación / baja de un integrante	49
10.3.3	R-07 — LLM alucina respuestas	49
10.4	Riesgos aceptados sin plan activo	50
11	Métricas del proyecto	51
11.1	Métricas de proceso (Scrum)	51
11.1.1	Velocity por sprint	51
11.1.2	Carry-over por sprint	51
11.1.3	Burn-down ideal vs real	52
11.2	Métricas de desarrollo	52
11.3	Métricas del producto en runtime	52
11.4	Calidad del LLM (golden set)	53
11.5	Métricas de costos	53
11.6	Métricas de gestión	54
12	Sprint 0 — Setup e Investigación	55
12.1	Objetivo del sprint	55
12.2	Duración	55

12.3 Tareas completadas	55
12.4 Entregables	55
12.5 Retrospectiva	56
13 Sprint 1 — Core chat	57
13.1 Objetivo del sprint	57
13.2 Duración	57
13.3 Tareas completadas	57
13.4 Entregables	57
13.5 Retrospectiva	57
14 Sprint 2 — RAG y carga de material	58
14.1 Objetivo	58
14.2 Duración	58
14.3 Tareas completadas	58
14.4 Entregables	58
14.5 Retrospectiva	58
15 Sprint 3 — Calidad y métricas	59
15.1 Objetivo	59
15.2 Duración	59
15.3 Tareas completadas	59
15.4 Entregables	59
15.5 Retrospectiva	59
16 Sprint 4 — MVP completo	60
16.1 Objetivo	60
16.2 Duración	60
16.3 Features entregadas	60
16.3.1 Feature A — Buscador semántico	60
16.3.2 Feature B — Chat multi-curso	60
16.3.3 Feature C — Streaming SSE	60
16.3.4 Feature D — Citas clickeables con preview	61
16.3.5 Feature E — Historial de conversaciones	61
16.3.6 Feature F — Quiz generator	61
16.3.7 Feature G — Detección de gaps del docente	61
16.4 Entregables del sprint	61
16.5 Retrospectiva	61
17 Arquitectura técnica	62
17.1 Visión general	62
17.1.1 Principios rectores	62
17.2 Diagrama de componentes	62
17.3 Patrón Hybrid PHP Proxy (ADR-001)	63
17.4 Flujo de una consulta del alumno	63
17.5 Pipeline RAG — indexación offline	64
17.6 Estados del proyecto y trayectoria post-MVP	65
18 Stack tecnológico	66

18.1	Tabla de componentes	66
18.2	Justificación de las decisiones clave	66
18.2.1	Monolito modular sobre microservicios	67
18.2.2	pgvector sobre ChromaDB	67
18.2.3	Arquitectura multi-provider LLM	67
18.2.4	Gemini 2.5 Flash en MVP, OpenAI en producción	67
18.2.5	HMAC SHA-256 en 3 capas	67
18.2.6	Privacy API con null_provider en MVP	68
18.3	Tecnologías evaluadas y descartadas	68
19	Modelo de datos	69
19.1	Visión general	69
19.2	Diagrama entidad-relación	69
19.3	Tablas	70
19.3.1	documents	70
19.3.2	chunks	71
19.3.3	chat_sessions	71
19.3.4	messages	72
19.3.5	unanswered_questions	72
19.4	Evolución del schema (migraciones Alembic)	73
19.5	Migración entre dimensiones de embeddings (post-MVP)	73
19.6	Tablas del plugin Moodle (local_nexusai_*)	73
20	API endpoints	75
20.1	Autenticación HMAC	75
20.2	Endpoints de salud	75
20.2.1	GET /health	75
20.2.2	GET /	76
20.3	Chat	76
20.3.1	POST /api/v1/chat/messages	76
20.3.2	POST /api/v1/chat/stream	76
20.3.3	POST /api/v1/chat/sessions/list	77
20.3.4	POST /api/v1/chat/sessions/messages	77
20.3.5	POST /api/v1/chat/echo	77
20.4	Documentos	77
20.4.1	POST /api/v1/documents	77
20.4.2	GET /api/v1/documents?course_id=N	78
20.4.3	GET /api/v1/documents/{document_id}	78
20.4.4	DELETE /api/v1/documents/{document_id}	78
20.5	Búsqueda semántica (Feature A)	78
20.5.1	POST /api/v1/search	78
20.6	Quiz generator (Feature F)	78
20.6.1	POST /api/v1/quiz/generate	78
20.7	Gaps del docente (Feature G)	79
20.7.1	POST /api/v1/gaps/list	79
20.8	Admin	80
20.8.1	GET /api/v1/admin/usage	80
20.8.2	GET /api/v1/courses/{course_id}/stats	80

21 Mockups y diseño de UI	81
21.1 Filosofía de diseño	81
21.2 Pantallas	81
21.2.1 1. Widget de chat flotante (alumno)	81
21.2.2 2. Pestañas Chat / Buscador / Quiz	81
21.2.3 3. Citas clickeables (Feature D)	81
21.2.4 4. Historial de conversaciones (Feature E)	81
21.2.5 5. Quiz en progreso (Feature F)	81
21.2.6 6. Vista del docente — NexusAI · Materials	81
21.2.7 7. Tab Gaps detectados (Feature G)	82
21.3 Sistema de design	82
21.3.1 Paleta de colores	82
21.3.2 Tipografía	82
21.3.3 Tokens CSS	82
22 Manual de instalación	83
22.1 Opción 1 — Instalar solo el plugin en un Moodle existente	83
22.1.1 Pasos	83
22.1.2 Permisos	84
22.2 Opción 2 — Stack completo local con Docker	84
22.2.1 Prerrequisitos	84
22.2.2 Pasos	84
22.2.3 Comandos útiles del día a día	85
22.3 Opción 3 — Plugin local + backend deployado en la nube	85
22.4 Troubleshooting	86
23 Manual de usuario	87
23.1 Manual del alumno	87
23.1.1 Acceder al asistente NexusAI	87
23.1.2 Hacer una consulta sobre el material del curso	87
23.1.3 Ver las fuentes que usó la IA	87
23.1.4 Consultar material de todos tus cursos a la vez	87
23.1.5 Buscar fragmentos sin usar el chat	87
23.1.6 Generar un quiz de práctica	87
23.1.7 Retomar una conversación anterior	87
23.2 Manual del docente	88
23.2.1 Acceder a NexusAI · Materials	88
23.2.2 Subir un PDF al material indexado	88
23.2.3 Eliminar un documento	88
23.2.4 Ver los “gaps detectados”	88
23.2.5 Configurar el plugin (administrador)	88
24 Architectural Decision Records (ADRs)	89
24.1 ¿Qué es un ADR?	89
24.2 ADR-001 — Backend Python como monolito modular	89
24.2.1 Contexto	89
24.2.2 Decisión	89
24.2.3 Consecuencias	90

24.3 ADR-002 — pgvector sobre PostgreSQL como única base	90
24.3.1 Contexto	90
24.3.2 Decisión	90
24.3.3 Alternativas descartadas	90
24.3.4 Consecuencias	90
24.4 ADR-003 — Arquitectura agnóstica de proveedor LLM	90
24.4.1 Contexto	91
24.4.2 Decisión	91
24.4.3 Consecuencias	91
24.5 ADR-004 — Gemini 2.5 Flash en MVP, GPT-4o-mini en producción	91
24.5.1 Contexto	91
24.5.2 Decisión	91
24.5.3 Consecuencias	91
24.6 ADR-005 — HMAC SHA-256 en 3 capas entre PHP y Python	92
24.6.1 Contexto	92
24.6.2 Decisión	92
24.6.3 Consecuencias	92
24.7 ADR-006 — Privacy API con <code>null_provider</code> en MVP	92
24.7.1 Contexto	93
24.7.2 Decisión	93
24.7.3 Consecuencias	93
24.8 Resumen	93
25 Estrategia de testing	94
25.1 Filosofía	94
25.2 Tests del backend Python (pytest)	94
25.2.1 Casos de prueba destacados	95
25.3 Tests del frontend (manual)	96
25.4 Tests del plugin Moodle (PHPUnit + Behat)	96
25.5 CI/CD — GitHub Actions	96
25.5.1 <code>backend-ci.yml</code>	96
25.5.2 <code>frontend-ci.yml</code>	96
25.5.3 <code>moodle-ci.yml</code>	96
25.5.4 Despliegue a Railway	97
25.6 Testing manual estructurado	97
25.6.1 Distribución de los casos por área	97
25.6.2 Estructura de cada caso	98
25.6.3 Ejemplos representativos	98
25.6.4 Estado de ejecución al cierre del MVP	99
25.7 Smoke tests rápidos pre-release	99
25.8 Cobertura actual	100
26 Deploy y producción	101
26.1 Resumen	101
26.2 Arquitectura de deploy	101
26.3 Backend en Railway	102
26.3.1 URL pública	102
26.3.2 Componentes deployados	102

26.3.3	Costos	102
26.3.4	Pipeline de autodeploy	103
26.3.5	Acceso al dashboard de Railway	103
26.4	Distribución del plugin Moodle	103
26.4.1	GitHub Release	103
26.4.2	Proceso de instalación en un Moodle de destino	104
26.4.3	Sub-misión al Moodle Plugin Directory (post-MVP)	104
26.5	Cómo funciona la demo de la defensa	104
26.6	Resiliencia y monitoreo	105
26.7	Plan de continuidad post-defensa	105
27	Retrospectivas	106
27.1	Retrospectiva por sprint	106
27.1.1	Sprint 0 — Setup e investigación	106
27.1.2	Sprint 1 — Core chat	106
27.1.3	Sprint 2 — RAG y carga de material	107
27.1.4	Sprint 3 — Calidad y métricas	107
27.1.5	Sprint 4 — MVP completo	107
27.2	Lecciones agregadas (retrospectiva global)	108
27.2.1	Lo que volveríamos a hacer	108
27.2.2	Lo que cambiaríamos	108
27.2.3	Lo que aprendimos sobre IA en educación	109
28	Conclusiones	110
28.1	Logros vs objetivos	110
28.2	Contribución académica	111
28.2.1	1. ¿Cómo se evita la alucinación del LLM en contextos académicos?	111
28.2.2	2. ¿Cómo se cierra el feedback loop alumno ↔ docente?	111
28.2.3	3. ¿Cómo se preservan la privacidad y soberanía de datos académicos?	111
28.3	Limitaciones del MVP	111
28.4	Trabajo futuro	112
28.4.1	Mejoras inmediatas (próximas 2-4 semanas post-defensa)	112
28.4.2	Líneas de investigación post-tesis	112
28.5	Reflexión final	112
29	Anexos	113
29.1	A. Glosario técnico	113
29.2	B. Bibliografía y recursos consultados	115
29.2.1	Sobre RAG y LLMs	115
29.2.2	Sobre Moodle	115
29.2.3	Sobre pgvector y bases vectoriales	115
29.2.4	Sobre administración de proyectos	115
29.3	C. Listado completo de historias de usuario	115
29.4	D. Listado de commits relevantes	116
29.5	E. Issues cerradas (GitHub)	116
29.6	F. Acceso al sistema demo	117
29.6.1	Backend en producción (Railway)	117
29.6.2	Credenciales para conectar un Moodle al backend	117

29.6.3 Demo presencial (laptop del equipo)	117
29.6.4 Repositorios públicos	118
29.7 G. Contacto del equipo	118

Chapter 1

Resumen ejecutivo

1.1 Problema

Las plataformas LMS (Learning Management Systems) como Moodle son fundamentales en la educación universitaria, pero en la práctica se usan como **repositorios estáticos**: los estudiantes descargan archivos y entregan trabajos, sin un acompañamiento real al proceso de aprendizaje.

Un relevamiento realizado a estudiantes de la Universidad Católica de Córdoba (UCC) identificó tres problemáticas recurrentes:

- **Dispersión de información** entre el campus virtual, WhatsApp y Google Drive — más del 70% de los estudiantes encuestados reportó dificultades para encontrar lo que necesita.
- **Dificultad para organizar el estudio** y planificar repasos antes de evaluaciones.
- **Uso pasivo del campus virtual**, sin interacción significativa con el material académico cargado por los docentes.

1.2 Solución propuesta

NexusAI es un plugin para Moodle que integra un **asistente académico basado en inteligencia artificial**, capaz de:

- Responder consultas en lenguaje natural sobre el contenido real de cada materia, usando Retrieval-Augmented Generation (RAG) sobre el material indexado del curso.
- Citar explícitamente el archivo y fragmento del que proviene cada respuesta — sin alucinaciones inventadas.
- Generar quizzes de práctica de opción múltiple a partir del material del docente.
- Detectar automáticamente qué temas consultan los alumnos que el material del curso no logra responder, generando feedback accionable para el docente.
- Mantener historial de conversación por sesión para que el alumno pueda retomar el estudio donde lo dejó.

La solución **no reemplaza Moodle ni al docente**: agrega una capa de inteligencia sobre la plataforma existente.

1.3 Diferenciadores técnicos

- **RAG auténtico con citas trazables:** cada respuesta del asistente enlaza al fragmento exacto del material del curso usado para generarla. Las pills clickeables del frontend expanden el chunk con su porcentaje de similaridad — trazabilidad como mitigación de alucinación.
- **Multi-provider LLM:** el backend abstrae el proveedor del modelo (Gemini en MVP, OpenAI en producción, Ollama local opcional) detrás de variables de entorno. Sin vendor lock-in.
- **Self-hosted, datos en la institución:** el patrón Hybrid PHP Proxy garantiza que la API key del LLM nunca llega al navegador y que los datos académicos no salen de la infraestructura de la institución.
- **Multi-curso opcional:** el alumno puede consultar el material de todos sus cursos a la vez, con respuestas que citan la materia de cada fragmento.
- **Feedback loop al docente:** el sistema mide su propia ceguera — registra preguntas que no pudo responder y se las muestra al docente para mejorar el material.

1.4 Estado del proyecto al cierre del MVP

- **7 features deployadas** en el Sprint 4 (junio 2026): buscador semántico, chat multi-curso, streaming SSE, citas clickeables, historial, quiz generator, detección de gaps.
- **Backend FastAPI** deployado en Railway con autodeploy desde GitHub Actions.
- **Plugin Moodle** distribuido como ZIP descargable en GitHub Release, compatible con Moodle 4.1 LTS hasta 4.5.
- **37 tests automatizados** en backend Python con cobertura ~80%.
- **Documentación técnica completa** — 6 ADRs, 5 diagramas Mermaid, 47 documentos de investigación previa, manuales de usuario y administrador.

1.5 Próximos pasos post-MVP

- Submission del plugin al Moodle Plugin Directory oficial para defendibilidad académica adicional.
- Implementación del módulo de analytics para el alumno (no solo para el docente).
- Switch de Gemini a OpenAI GPT-4o-mini en producción.
- Pilotos con cursos reales de la UCC para recolectar métricas de uso e impacto pedagógico.

Chapter 2

Introducción y contexto

2.1 Contexto

La adopción de inteligencia artificial generativa en la educación universitaria avanza más rápido que la integración formal de estas herramientas en las plataformas educativas. Mientras los alumnos usan ChatGPT y otros asistentes generales para resolver dudas académicas, las plataformas LMS institucionales como Moodle siguen funcionando como repositorios documentales, sin integración con la IA.

Esto genera dos problemas estructurales:

1. **Los docentes pierden visibilidad** sobre qué preguntan los alumnos sobre su material, porque las consultas suceden afuera del LMS.
2. **Los alumnos reciben respuestas genéricas** de modelos entrenados sobre internet abierto, no sobre el material específico de su curso — con riesgo de respuestas plausibles pero incorrectas en el contexto de esa materia particular.

2.2 Motivación

El equipo del proyecto realizó un relevamiento informal entre estudiantes de la Universidad Católica de Córdoba (UCC) sobre el uso del campus virtual. Los hallazgos motivaron el proyecto:

- Más del 70% reportó **dificultades para encontrar información** que estaba dispersa entre el campus, WhatsApp grupales y Google Drive personales.
- La mayoría declaró usar ChatGPT u otros asistentes generales como primer recurso ante una duda, **antes** que el campus virtual.
- Los docentes encuestados expresaron interés en herramientas que les permitan ver qué temas resultan más confusos para sus alumnos.

A partir de ese diagnóstico, el equipo propuso un plugin de Moodle que:

- Mantiene a los alumnos en el campus virtual (donde está el material real).
- Responde sobre **el material específico del curso**, no sobre internet.
- Genera datos accionables para el docente.

2.3 Objetivos del proyecto

2.3.1 Objetivo general

Desarrollar un plugin para Moodle que incorpore un asistente académico basado en inteligencia artificial, capaz de responder consultas en lenguaje natural sobre el contenido real de cada materia, generar ejercicios de práctica personalizados y proveer herramientas de analytics al docente, mejorando la experiencia educativa dentro del aula virtual de la UCC.

2.3.2 Objetivos específicos

- Identificar y documentar las problemáticas del uso actual del campus virtual mediante relevamiento a estudiantes y docentes de la UCC.
- Diseñar e implementar un plugin tipo `local` para Moodle que se integre al entorno existente sin modificarlo.
- Desarrollar un prototipo funcional (MVP) con un asistente conversacional basado en técnicas de Retrieval-Augmented Generation (RAG) e integración con modelos de lenguaje de gran escala, validado con usuarios reales.
- Implementar un sistema de generación de quizzes, flashcards y ejercicios de práctica con corrección automática e IA.
- Proveer al docente un dashboard de analytics sobre las consultas de sus alumnos y herramientas de detección de gaps en el material.
- Evaluar el impacto de la solución mediante pruebas con usuarios reales y métricas de calidad de respuesta.
- Documentar el proceso completo de diseño, desarrollo e implementación para su presentación y defensa ante la cátedra.

2.4 Alcance de este documento

Este documento es la **entrega final** del proyecto correspondiente al Proyecto Integrador de la carrera Ingeniería en Sistemas (UCC, 2026). Cubre:

- La gestión completa del proyecto desde el Sprint 0 (setup e investigación) hasta el Sprint 4 (cierre del MVP).
- La documentación técnica del producto: arquitectura, stack, modelo de datos, API, decisiones arquitectónicas, testing y deploy.
- Los manuales de instalación y de uso para alumnos y docentes.
- Análisis de costos, riesgos y métricas del proyecto.
- Retrospectivas por sprint y conclusiones del equipo.

Cada capítulo es autocontenido y referencia material complementario del repositorio público del proyecto en GitHub.

2.5 Equipo

Integrante	Rol
Santiago Tricherri	Project Manager · Backend & AI Developer
Delfina Salinas	Scrum Master · Frontend & RAG Developer

2.6 Universidad e institución

- **Universidad:** Universidad Católica de Córdoba
- **Facultad:** Facultad de Ingeniería
- **Carrera:** Ingeniería en Sistemas
- **Año académico:** 2026

Chapter 3

Alcance del proyecto

3.1 Producto esperado

Un plugin funcional instalable en Moodle 4.1–4.5 que permita a los estudiantes consultar el contenido de su materia en lenguaje natural y recibir respuestas contextualizadas basadas en los materiales reales del curso, validado mediante pruebas con usuarios reales de la UCC.

3.2 In scope — funcionalidades del MVP

3.2.1 Plugin Moodle

- Desarrollo del plugin tipo `local` instalable en Moodle 4.1 LTS hasta 4.5.
- Integración con el entorno existente sin modificar el core de Moodle.
- Compatibilidad con el sistema de capabilities estándar (`local/nexusai:use`, `:manage`, `:viewanalytics`).
- Privacy API declarada (`null_provider` en MVP).
- Soporte de internacionalización (`es / en`).

3.2.2 Backend FastAPI

- Implementación del backend Python con integración a modelos de lenguaje de gran escala (LLM) a través de una API de IA generativa.
- Sistema RAG con base de datos vectorial (`pgvector` sobre PostgreSQL) para indexación y búsqueda semántica de materiales.
- Auth HMAC SHA-256 server-to-server entre plugin y backend.
- Rate limiting por usuario.
- Logging estructurado JSON.

3.2.3 Interfaz de chat

- Widget de chat embebido en las páginas del curso, accesible vía FAB flotante.
- Streaming token-por-token de las respuestas del LLM (Server-Sent Events).
- Citas clickeables con preview del fragmento usado.

- Historial de conversaciones del alumno por sesión.

3.2.4 Gestión de material por parte del docente

- Carga de archivos PDF al backend con indexación automática asíncrona.
- Visualización del estado de cada documento (pending $\square$ indexing $\square$ indexed | error).
- Borrado de documentos con cascade sobre los chunks vectorizados.
- Detección de uploads duplicados por hash SHA-256.

3.2.5 Funcionalidades complementarias del MVP

- Buscador semántico (retrieval puro sin LLM) sobre el material indexado.
- Chat multi-curso opcional con citas que incluyen el nombre de la materia.
- Quiz generator de opción múltiple a partir del material del curso, con feedback inmediato + explicación + archivo fuente.
- Dashboard básico para el docente con detección de gaps (preguntas no respondidas por el material).

3.2.6 Calidad y operación

- Tests automatizados en backend (cobertura $\geq 70\%$).
- CI/CD en GitHub Actions para backend, frontend y plugin PHP.
- Deploy automático del backend a Railway desde la rama main.
- Distribución del plugin como ZIP descargable en GitHub Releases.

3.2.7 Validación

- Realización de pruebas con usuarios reales y recolección de feedback.

3.3 Out of scope — explícitamente descartado del MVP

Los siguientes ítems fueron evaluados y **conscientemente excluidos** del alcance del MVP para no diluir el foco. Algunos están planificados para post-MVP:

Ítem fuera de alcance	Razón
Soporte multi-institución (multi-tenant nativo)	Cada institución debe hostear su propio backend. Multi-tenant con aislamiento estricto requiere <code>tenant_id</code> en cada tabla y auth más compleja.
Integración con WhatsApp	Fuera del foco académico. Reduce el problema de dispersión hacia adentro de Moodle.
Integración con sistemas administrativos de la universidad	Excede el alcance de un MVP. Requiere acuerdos institucionales.
Generación o análisis de contenido multimedia (audio, video, imágenes)	Limitado a texto en MVP por restricciones de costo del LLM y complejidad.

Ítem fuera de alcance	Razón
Calendario, alertas y notificaciones	Funcionalidad nativa de Moodle. Duplicarla no agrega valor diferencial.
Foros mejorados con IA	Planificado post-MVP. Requiere integración con módulo mod_forum de Moodle.
Study Planner avanzado (planes personalizados)	Planificado post-MVP. Requiere modelar progreso del alumno, no solo responder consultas.
Resúmenes automáticos de materiales completos	Planificado post-MVP. El quiz generator cubre parcialmente esa necesidad.
Tutoría conversacional con seguimiento longitudinal	Fuera de alcance. Requiere infraestructura más compleja de modelado del alumno.
Análisis automático de sesgo en el material	Tema sensible, fuera del MVP. Mencionado como línea futura.

3.4 Usuarios objetivo

3.4.1 Rol primario: alumno

- Universitario de UCC en cualquier carrera que use Moodle.
- Edad 18-30 típica.
- Acceso a la plataforma vía navegador (desktop o mobile).
- Capability requerida: local/nexusai:use en el curso.

3.4.2 Rol secundario: docente

- Profesor o auxiliar docente con responsabilidad de gestionar el material de un curso.
- Capability requerida: local/nexusai:manage en el curso.

3.4.3 Rol terciario: administrador de Moodle

- Persona responsable de instalar el plugin y configurar el endpoint del backend, la API key y el shared secret.

3.5 Restricciones del proyecto

Restricción	Valor
Equipo	2 personas (Santiago Tricherri + Delfina Salinas)
Presupuesto operativo	\$0 USD reales (uso de tier gratuito Gemini + GitHub Student Pack)
Versión Moodle soportada	4.1 LTS hasta 4.5
Idiomas	Español + inglés
Tipo de archivo soportado en MVP	PDF (DOCX y TXT están en el roadmap)

Restricción	Valor
Tamaño máximo de archivo	20 MB por documento
LLM en MVP	Gemini 2.5 Flash (tier gratuito)
LLM en producción	OpenAI GPT-4o-mini

Chapter 4

Análisis de requerimientos

4.1 Convenciones

Cada requerimiento tiene:

- Un **ID único** (RF-NN para funcionales, RNF-NN para no funcionales).
- Una **prioridad** (Alta / Media / Baja) en el contexto del MVP.
- Un **sprint asignado** donde se implementó.

4.2 Requerimientos funcionales

4.2.1 Asistente conversacional RAG

ID	Requerimiento	Prioridad	Sprint
RF-01	Como alumno, debo poder hacer una consulta en lenguaje natural sobre el material del curso desde un widget flotante en el campus virtual.	Alta	1
RF-02	El asistente debe responder usando exclusivamente el material indexado del curso, citando explícitamente los archivos fuente.	Alta	2

ID	Requerimiento	Prioridad	Sprint
RF-03	Si la pregunta no se puede responder con el material disponible, el sistema debe admitirlo explícitamente sin inventar respuestas.	Alta	2
RF-04	El alumno debe poder ver el fragmento exacto del material que el sistema usó para responder, con su porcentaje de similaridad.	Alta	4
RF-05	La respuesta del LLM debe aparecer token-por-token (streaming) para reducir la latencia perceived.	Media	4
RF-06	El alumno debe poder retomar conversaciones anteriores desde un historial accesible.	Media	4
RF-07	El alumno debe poder activar opcionalmente el modo multi-curso, donde el asistente consulta material de todos sus cursos enrollados.	Media	4

4.2.2 Buscador semántico

ID	Requerimiento	Prioridad	Sprint
RF-08	El alumno debe poder buscar fragmentos del material del curso por similaridad semántica, sin pasar por el LLM.	Media	4

ID	Requerimiento	Prioridad	Sprint
RF-09	Cada resultado del buscador debe mostrar el archivo, el fragmento de texto, y el porcentaje de similaridad.	Media	4

4.2.3 Quiz generator

ID	Requerimiento	Prioridad	Sprint
RF-10	El alumno debe poder generar un quiz de práctica de opción múltiple desde el material del curso.	Media	4
RF-11	El alumno debe poder elegir un tema específico o dejar el campo vacío para variedad de temas.	Media	4
RF-12	Si el tema solicitado no está en el material, el sistema debe avisar al alumno (no generar quiz aleatorio engañoso).	Media	4
RF-13	Cada pregunta debe tener exactamente 4 opciones, una correcta y tres distractores plausibles.	Media	4
RF-14	El alumno debe recibir feedback inmediato después de cada respuesta con explicación y archivo fuente.	Media	4

4.2.4 Gestión de material (rol docente)

ID	Requerimiento	Prioridad	Sprint
RF-15	Como docente, debo poder subir archivos PDF al curso para que sean indexados por el asistente.	Alta	2
RF-16	El sistema debe extraer texto, chunking y embeddings de cada archivo de manera asíncrona.	Alta	2
RF-17	El docente debe ver el estado de cada documento (pending / indexing / indexed / error).	Alta	2
RF-18	El docente debe poder borrar un documento. El borrado debe propagarse en cascada a los chunks vectorizados.	Alta	2
RF-19	El sistema debe detectar uploads duplicados por hash SHA-256 y evitar re-indexar el mismo archivo.	Media	4

4.2.5 Feedback loop al docente (detección de gaps)

ID	Requerimiento	Prioridad	Sprint
RF-20	El sistema debe registrar automáticamente las preguntas que el material no pudo responder.	Media	4
RF-21	El docente debe poder ver un reporte agregado de gaps de su curso, ordenado por frecuencia.	Media	4

ID	Requerimiento	Prioridad	Sprint
RF-22	El reporte de gaps debe filtrarse por ventana temporal (7 días / 30 días / 90 días / 365 días).	Baja	4

4.3 Requerimientos no funcionales

4.3.1 Seguridad

ID	Requerimiento	Justificación
RNF-01	La API key del LLM no debe llegar nunca al navegador del alumno.	Patrón Hybrid PHP Proxy (ADR-001), HMAC server-to-server (ADR-005). ADR-005.
RNF-02	Toda comunicación entre el plugin Moodle y el backend debe firmarse con HMAC SHA-256 con 3 capas (Bearer + firma + nonce Redis).	
RNF-03	El backend debe rechazar requests con firma inválida, timestamp expirado (>5 min) o nonce reusado (replay).	Anti-replay.
RNF-04	El plugin debe validar capability <code>local/nexusai:use</code> antes de enviar requests al backend.	Aislamiento por curso.
RNF-05	El backend debe validar ownership de las sesiones de chat (un usuario no puede leer sesiones ajenas).	Privacy.

4.3.2 Performance

ID	Requerimiento	Valor objetivo
RNF-06	Latencia al primer token de respuesta del LLM (streaming)	< 2 s
RNF-07	Latencia de respuesta completa del chat (sin streaming)	< 8 s
RNF-08	Latencia del buscador semántico (retrieval puro)	< 1 s

ID	Requerimiento	Valor objetivo
RNF-09	Tiempo de indexación de un PDF de 50 páginas	< 60 s

4.3.3 Compatibilidad

ID	Requerimiento
RNF-10	El plugin debe instalarse en Moodle 4.1 LTS hasta 4.5 sin requerir composer install ni dependencias externas.
RNF-11	El widget de chat debe funcionar en los navegadores principales (Chrome, Firefox, Safari, Edge) en versiones actuales.
RNF-12	El backend debe correr sobre Python 3.11+ con PostgreSQL 16+ y pgvector 0.3.5+.

4.3.4 Privacidad y soberanía de datos

ID	Requerimiento
RNF-13	Los datos académicos (mensajes, documentos, embeddings) deben vivir en una base de datos controlada por la institución que hospeda el backend.
RNF-14	El plugin debe declarar correctamente su Privacy API según la versión de Moodle (null_provider en MVP, metadata\provider planificado).
RNF-15	Multi-provider LLM: la institución debe poder cambiar el proveedor del LLM sin modificar código (variable de entorno).

4.3.5 Mantenibilidad

ID	Requerimiento
RNF-16	Cobertura de tests automatizados del backend Python $\geq 70\%$.
RNF-17	CI/CD en GitHub Actions para backend, frontend y plugin PHP.
RNF-18	Decisiones arquitectónicas documentadas como ADRs en docs/adr/.

ID	Requerimiento
RNF-19	Cada feature documentada con: ADR (si aplica), comentarios en código, issue en GitHub con criterios de aceptación.

4.3.6 Distribución

ID	Requerimiento
RNF-20	El plugin debe distribuirse como ZIP descargable instalable vía Site administration □ Plugins □ Install plugins.
RNF-21	Cada release debe estar etiquetada en GitHub con changelog y la versión <code>version.php</code> actualizada.

Chapter 5

Historias de usuario y criterios de aceptación

5.1 Convenciones

Las historias de usuario siguen el formato estándar de Scrum:

Como [rol], **quiero** [funcionalidad], **para** [beneficio].

Cada historia tiene asociados:

- Una **épica** padre (agrupación temática).
- Una **prioridad** (Alta / Media / Baja).
- Una **estimación en story points** (escala Fibonacci: 1, 2, 3, 5, 8).
- Un **responsable** asignado.
- Un **sprint** donde se completó.
- **Criterios de aceptación** en formato Dado/Cuando/Entonces.

5.2 Resumen agregado del backlog

Categoría	Cantidad de historias	Story points totales
Investigación	14	~38
Setup	10	~22
Backend Python	14	~85
Plugin Moodle	8	~40
Frontend React	8	~26
Gestión Scrum (sprint planning, reviews, retros, dailies)	10	~10
Features Sprint 4 (A-G)	7 + sub-tareas	~70
Total MVP	70+ historias	~290 SP

El backlog completo se incluye como anexo en el archivo NexusAI – Backlog Completo.xlsx del pendrive de entrega. A continuación se documentan las épicas principales con ejemplos representativos.

5.3 Épica 1 — Asistente conversacional con RAG

Permitir al alumno hacer consultas en lenguaje natural sobre el material del curso y recibir respuestas contextualizadas con citas trazables.

5.3.1 HU-001 — Chat básico del alumno

Como alumno, **quiero** abrir un widget de chat desde mi curso de Moodle y hacer una pregunta en lenguaje natural, **para** obtener respuestas inmediatas sobre el material de mi materia.

Criterios de aceptación:

1. **Dado** que estoy logueado en un curso de Moodle con material indexado, **cuando** abro el widget flotante (FAB), **entonces** veo el panel de chat con un input y mensaje de bienvenida.
2. **Dado** el panel de chat abierto, **cuando** escribo una pregunta y presiono Enter, **entonces** mi mensaje aparece en pantalla y se envía al backend.
3. **Dado** una pregunta enviada, **cuando** el backend responde, **entonces** la respuesta del asistente aparece en una burbuja diferenciada (no mi propia).

5.3.2 HU-002 — Citas trazables a la fuente

Como alumno, **quiero** ver de qué archivo y fragmento proviene cada respuesta del asistente, **para** poder verificar y profundizar en el material original.

Criterios de aceptación:

1. **Dado** una respuesta del asistente que usa material indexado, **cuando** veo el mensaje, **entonces** debajo aparece una sección “Fuentes:” con pills (chips) por cada archivo citado.
2. **Dado** las pills de fuentes, **cuando** hago click en una, **entonces** se expande inline mostrando el fragmento exacto del archivo y su porcentaje de similaridad con mi pregunta.
3. **Dado** que el sistema no encontró material relevante, **cuando** veo la respuesta, **entonces** la respuesta admite explícitamente que no puede responder con el material disponible.

5.3.3 HU-003 — Streaming token-por-token

Como alumno, **quiero** ver la respuesta del asistente aparecer palabra por palabra mientras el LLM la genera, **para** no esperar varios segundos en blanco antes de leer algo.

Criterios de aceptación:

1. **Dado** una pregunta enviada al backend, **cuando** el primer token está disponible, **entonces** aparece en pantalla en menos de 2 segundos.
2. **Dado** que la respuesta se está streameando, **cuando** veo el texto, **entonces** un caret azul parpadeante (▯) aparece al final indicando que sigue llegando contenido.
3. **Dado** que el stream termina, **cuando** la respuesta está completa, **entonces** el caret desaparece y aparecen las pills de fuentes.

5.4 Épica 2 — Gestión de material por parte del docente

Permitir al docente subir, monitorear y eliminar el material que el asistente usará para responder.

5.4.1 HU-010 — Upload de PDFs por el docente

Como docente, **quiero** poder subir archivos PDF al curso desde una página dedicada, **para** que el asistente pueda usarlos para responder a mis alumnos.

Criterios de aceptación:

1. **Dado** que tengo capability `local/nexusai:manage` en el curso, **cuando** ingreso a NexusAI · Materials, **entonces** veo una zona de drag-and-drop para subir archivos.
2. **Dado** que arrastro un PDF válido, **cuando** se sube, **entonces** aparece en la tabla con estado `pending` y luego `indexing`.
3. **Dado** que el archivo está siendo indexado, **cuando** la indexación termina exitosamente, **entonces** su estado pasa a `indexed` y queda disponible para consultas.
4. **Dado** que el archivo no se puede procesar (PDF escaneado sin OCR, corrupto, etc.), **cuando** termina el procesamiento, **entonces** su estado es `error` con mensaje explicativo en tooltip.

5.4.2 HU-011 — Detección de duplicados

Como docente, **quiero** que el sistema detecte si subo dos veces el mismo archivo, **para** no duplicar material indexado.

Criterios de aceptación:

1. **Dado** que un archivo ya está indexado en el curso, **cuando** subo el mismo contenido (mismo SHA-256), **entonces** el sistema reconoce el duplicado y devuelve el documento existente sin re-indexar.

5.5 Épica 3 — Buscador semántico (Feature A)

Búsqueda pura del material sin pasar por el LLM, útil para encontrar fragmentos rápidamente.

5.5.1 HU-020 — Buscar fragmentos por similitud

Como alumno, **quiero** buscar fragmentos del material del curso con una query en lenguaje natural sin esperar al LLM, **para** encontrar rápido en qué archivo está un tema.

Criterios de aceptación:

1. **Dado** que abro la pestaña “Buscador” del widget, **cuando** escribo una consulta y presiono buscar, **entonces** recibo en menos de 1 segundo una lista de fragmentos ordenados por similitud.
2. **Dado** los resultados, **cuando** miro cada uno, **entonces** veo el nombre del archivo, el texto del fragmento (~400 chars), y el porcentaje de similitud coloreado (verde > 75%, naranja 50-75%, gris < 50%).
3. **Dado** que el curso no tiene material indexado, **cuando** busco algo, **entonces** veo el mensaje “No encontré fragmentos relacionados”.

5.6 Épica 4 — Quiz generator (Feature F)

Generación de preguntas de opción múltiple para autoevaluación.

5.6.1 HU-030 — Generar quiz desde el material

Como alumno, **quiero** generar un quiz de opción múltiple sobre el material del curso, **para** repasar antes de un parcial.

Criterios de aceptación:

1. **Dado** que abro la pestaña “Quiz”, **cuando** elijo cantidad (3/5/7/10) y opcionalmente un tema, **entonces** click en “Generar quiz” inicia la generación.
2. **Dado** una pregunta generada, **cuando** la veo, **entonces** tiene exactamente 4 opciones (A/B/C/D) y un botón “Verificar”.
3. **Dado** que selecciono una opción y presiono verificar, **cuando** la corrección aparece, **entonces** veo si acerté + explicación + archivo fuente.
4. **Dado** que termino todas las preguntas, **cuando** veo la pantalla final, **entonces** muestra el score (X/N), el porcentaje, e icono según rango (trofeo $\geq 80\%$, pulgar 50-80%, libro $< 50\%$).

5.6.2 HU-031 — Tema sin material disponible

Como alumno, **quiero** que el sistema me avise si pido un quiz sobre un tema que no está en el material, **para** no recibir un quiz aleatorio engañoso.

Criterios de aceptación:

1. **Dado** que escribo un tema en el campo “Tema”, **cuando** ese tema no tiene chunks relevantes en el material (similitud < 0.4 o menos de 3 matches), **entonces** veo un mensaje claro pidiendo otro tema o dejar el campo vacío.

5.7 Épica 5 — Historial de conversaciones (Feature E)

Permitir al alumno retomar conversaciones previas.

5.7.1 HU-040 — Lista de sesiones previas

Como alumno, **quiero** ver mis conversaciones anteriores y retomar una específica, **para** continuar el estudio donde lo dejé.

Criterios de aceptación:

1. **Dado** que tengo sesiones de chat previas, **cuando** click en el botón  del header, **entonces** se abre un dropdown con la lista ordenada por última actividad.
2. **Dado** la lista, **cuando** miro cada ítem, **entonces** veo preview del primer mensaje, tiempo relativo (ej: “hace 2 horas”), y cantidad de mensajes.
3. **Dado** que selecciono una sesión, **cuando** click, **entonces** se cargan sus mensajes en el chat y puedo continuar la conversación enviando nuevas preguntas.

5.8 Épica 6 — Multi-curso (Feature B)

Permitir consultas que crucen el material de varios cursos del alumno.

5.8.1 HU-050 — Activar modo multi-curso

Como alumno enrollado en varios cursos, **quiero** poder hacer consultas que crucen el material de todas mis materias, **para** encontrar conexiones entre temas relacionados.

Criterios de aceptación:

1. **Dado** que estoy en el chat, **cuando** click en el botón  del header, **entonces** cambia a  y el status indica “Activo · todos tus cursos”.
2. **Dado** el modo multi-curso activo, **cuando** hago una pregunta, **entonces** la respuesta usa material de todos mis cursos enrollados y las pills citan la materia de cada fragmento.

5.9 Épica 7 — Detección de gaps del docente (Feature G)

Feedback loop pedagógico automático.

5.9.1 HU-060 — Ver gaps detectados

Como docente, **quiero** ver una lista de preguntas que mis alumnos hicieron y que el material no pudo responder bien, **para** identificar qué temas debo agregar al curso.

Criterios de aceptación:

1. **Dado** que tengo capability `local/nexusai:manage` en el curso, **cuando** ingreso a NexusAI · Materials  tab “Gaps detectados”, **entonces** veo una lista de preguntas agrupadas por similitud.
2. **Dado** la lista, **cuando** miro cada item, **entonces** veo la pregunta, cuántas veces se preguntó, la fecha de la última vez, y la similaridad promedio del material.
3. **Dado** que filtro por ventana temporal, **cuando** cambio entre 7 días / 30 días / 90 días / 365 días, **entonces** la lista se actualiza.

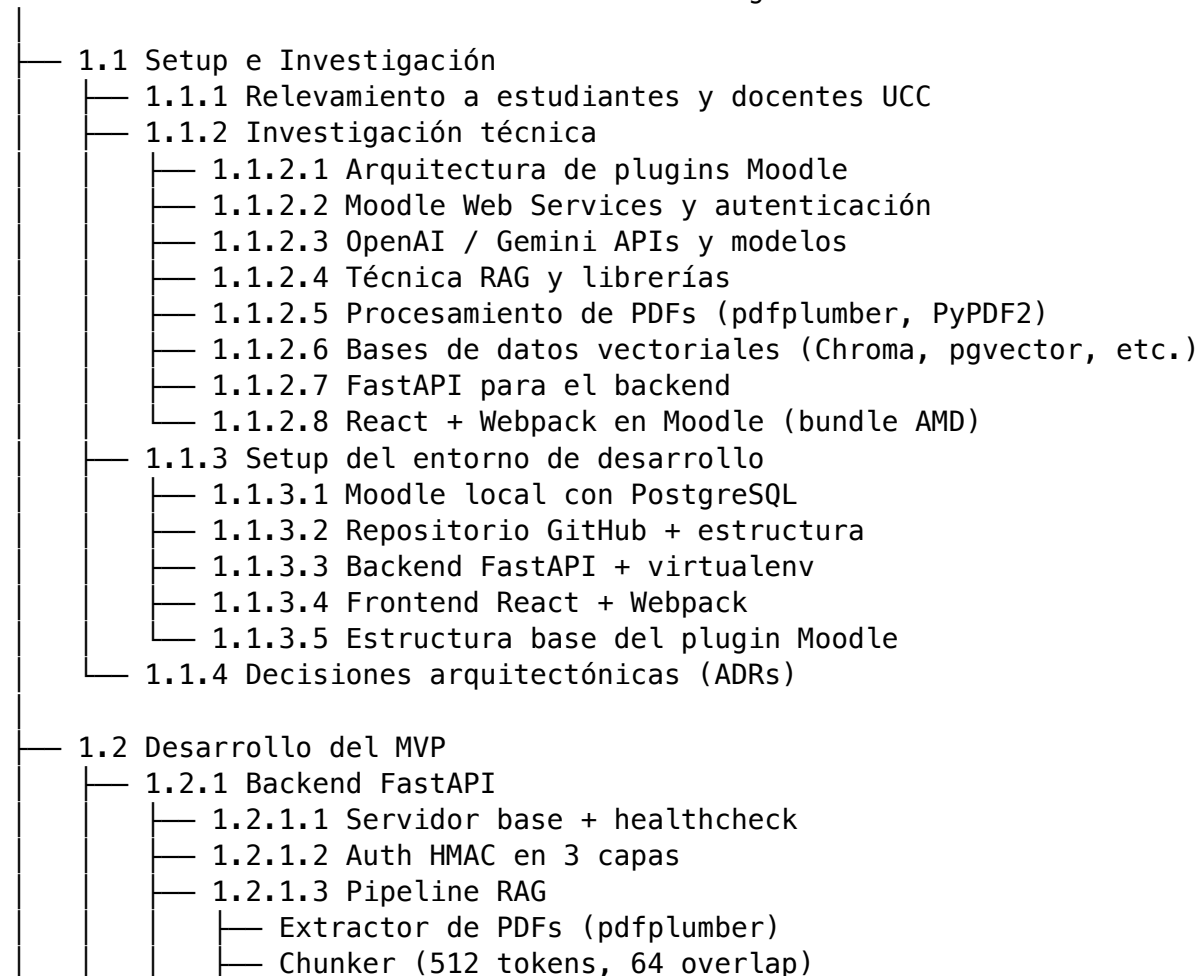
Chapter 6

Work Breakdown Structure (WBS)

6.1 Descomposición jerárquica del proyecto

El proyecto NexusAI se descompone en cinco entregables principales, desglosados a tres niveles de detalle:

1. NexusAI – Asistente académico con IA integrado en Moodle



	— EmbeddingProvider (Gemini Matryoshka)
	— Retriever con pgvector + HNSW
	— 1.2.1.4 Endpoints de documents (upload, list, delete)
	— 1.2.1.5 Endpoint de chat (sync + streaming SSE)
	— 1.2.1.6 LLMPProvider con multi-provider
	— 1.2.1.7 Rate limiting + logging estructurado
	— 1.2.1.8 Migraciones Alembic
— 1.2.2	Plugin Moodle
	— 1.2.2.1 Estructura base (version.php, install.xml, lang/)
	— 1.2.2.2 Capabilities (use, manage, viewanalytics)
	— 1.2.2.3 External Functions (chat_send, document_upload, etc.)
	— 1.2.2.4 backend_client.php (cURL + HMAC)
	— 1.2.2.5 Hooks Moodle 4.4+ y callback legacy 4.1–4.3
	— 1.2.2.6 Página documents.php para docente
	— 1.2.2.7 Endpoint chat_stream.php (SSE proxy)
	— 1.2.2.8 Privacy API (null_provider)
— 1.2.3	Bundle React
	— 1.2.3.1 Webpack config con output AMD
	— 1.2.3.2 Componentes del chat (App, MessageBubble, Input)
	— 1.2.3.3 Componentes del docente (DocumentsManager, GapsPanel)
	— 1.2.3.4 Componentes Sprint 4 (SearchPanel, QuizPanel, HistoryDropdown)
	— 1.2.3.5 Sistema de iconos lucide-style
— 1.2.4	Features Sprint 4
	— 1.2.4.1 Feature A – Buscador semántico
	— 1.2.4.2 Feature B – Chat multi-curso
	— 1.2.4.3 Feature C – Streaming SSE
	— 1.2.4.4 Feature D – Citas clickeables con preview
	— 1.2.4.5 Feature E – Historial de conversaciones
	— 1.2.4.6 Feature F – Quiz generator
	— 1.2.4.7 Feature G – Detección de gaps del docente
— 1.3	Testing y QA
	— 1.3.1 Tests unitarios backend (pytest)
	— test_hmac.py (10 tests)
	— test_chunker.py (13 tests)
	— test_retriever.py (8 tests)
	— test_extractor.py (6 tests)
	— 1.3.2 Tests de integración
	— test_documents_router.py
	— test_pipeline.py
	— test_providers.py
	— 1.3.3 CI/CD en GitHub Actions
	— backend-ci.yml (lint + tests)
	— frontend-ci.yml (build React)
	— moodle-ci.yml (PHP lint + Moodle checker)
	— deploy.yml (autodeploy a Railway)
	— 1.3.4 Testing manual E2E
	— Smoke tests por feature

└─	Validación en Moodle real
└─	1.4 Deploy y distribución
└─	└─ 1.4.1 Backend en Railway con autodeploy desde main
└─	└─ 1.4.2 Plugin como GitHub Release (ZIP descargable)
└─	└─ 1.4.3 Moodle público para defensa (DigitalOcean o Oracle Cloud)
└─	└─ 1.4.4 Submission al Moodle Plugin Directory (post-MVP)
└─	└─ 1.4.5 Documentación de instalación y troubleshooting
└─	1.5 Documentación y entrega final
└─	└─ 1.5.1 Documentación técnica
└─	└─ README.md
└─	└─ 6 ADRs (decisiones arquitectónicas)
└─	└─ 5 diagramas Mermaid (architecture, deployment, ER, RAG flow, sequence)
└─	└─ 47 documentos de investigación
└─	└─ CORRER_PROYECTO.md
└─	└─ 1.5.2 Documentación académica (este documento)
└─	└─ Resumen ejecutivo, introducción, alcance
└─	└─ Análisis de requerimientos
└─	└─ Historias de usuario
└─	└─ WBS, cronograma, estimaciones, costos, riesgos, métricas
└─	└─ Documentación por sprint
└─	└─ Documentación del producto (arquitectura, stack, modelo de datos, API)
└─	└─ Manuales (instalación, usuario)
└─	└─ ADRs sintetizados, testing, deploy
└─	└─ Retrospectivas, conclusiones, anexos
└─	1.5.3 Presentación de defensa (15 minutos)

6.2 Cargas de trabajo por entregable

Nivel 1 — Entregable	Story Points totales	% del total
1.1 Setup e Investigación	~60	21%
1.2 Desarrollo del MVP	~180	62%
1.3 Testing y QA	~20	7%
1.4 Deploy y distribución	~10	3%
1.5 Documentación y entrega	~20	7%
Total	~290 SP	100%

Los story points se asignaron usando escala Fibonacci (1, 2, 3, 5, 8) en planning poker con el equipo. El detalle por historia se encuentra en NexusAI – Backlog Completo.xlsx.

Chapter 7

Cronograma del proyecto

7.1 Fases del proyecto

El proyecto se gestiona con metodología **Scrum** adaptada al contexto académico. Cada sprint tiene un objetivo claro y termina con una review + retrospectiva. Las dailies son **asíncronas** vía GitHub Projects (actualización diaria de estado: qué hice, qué haré, bloqueos).

Fase	Fechas	Duración	Entregable clave	Estado
Setup e Investigación	09 Abr – 22 Abr 2026	2 semanas	Entorno + arquitectura definida	☐
Sprint 1	23 Abr – 06 May 2026	2 semanas	Plugin base + API Python + React	☐
Sprint 2	07 May – 20 May 2026	2 semanas	RAG completo + chat end-to-end	☐
Sprint 3	21 May – 27 May 2026	1 semana	Integración Moodle + contenido docente	☐
Sprint 4 — MVP ☐	28 May – 01 Jun 2026	1 semana	MVP entregado el 1 de junio	☐
Documentación MVP	02 Jun – 15 Jun 2026	2 semanas	Informe + presentación + demo	☐ en curso
Post-MVP (analytics, foros, planner)	Jun – Ago 2026	8 semanas	Sistema completo	☐ planificado
Informe parcial PI	antes del 3 Nov 2026	—	Check 2: diagnóstico + MT + objetivos	☐ planificado
Ajustes finales + PPT	Ene – 7 Feb 2027	5 semanas	Last check + ensayo defensa	☐ planificado

Fase	Fechas	Duración	Entregable clave	Estado
Defensa final ☐	20 – 27 Feb 2027	1 semana	Proyecto completo	☐ planificado

7.2 Diagrama Gantt

gantt

```

title NexusAI – Cronograma del proyecto
dateFormat YYYY-MM-DD
axisFormat %b %Y

```

```

section Setup

```

```

Setup e Investigación           :done, s0, 2026-04-09, 14d

```

```

section Desarrollo MVP

```

```

Sprint 1 – Core chat           :done, s1, 2026-04-23, 14d

```

```

Sprint 2 – RAG                  :done, s2, 2026-05-07, 14d

```

```

Sprint 3 – Calidad              :done, s3, 2026-05-21, 7d

```

```

Sprint 4 – MVP completo         :done, s4, 2026-05-28, 5d

```

```

MVP entregado                   :milestone, m1, 2026-06-01, 0d

```

```

section Cierre académico

```

```

Documentación MVP              :active, d1, 2026-06-02, 14d

```

```

Post-MVP                       :      d2, 2026-06-16, 70d

```

```

Informe parcial PI             :milestone, m2, 2026-11-03, 0d

```

```

Ajustes y PPT                  :      d3, 2027-01-15, 23d

```

```

Defensa final                   :milestone, m3, 2027-02-27, 0d

```

7.3 Detalle por sprint

7.3.1 Sprint 0 — Setup e Investigación (9 - 22 Abr 2026)

Duración: 2 semanas. **Capacidad:** ~120 horas-persona (2 personas × 60 horas). **Objetivo:** Investigar tecnologías clave y montar el entorno de desarrollo.

Entregables:

- 47 documentos de investigación cubriendo Moodle, RAG, OpenAI/Gemini, pgvector, FastAPI, React+Webpack, seguridad.
- Entorno de desarrollo funcional (Moodle local + PostgreSQL + FastAPI + React).
- Repositorio GitHub con estructura y CI básico.
- 6 ADRs definidos.
- Backlog del MVP cargado en GitHub Projects.

7.3.2 Sprint 1 — Core chat (23 Abr - 6 May 2026)

Duración: 2 semanas. **Capacidad:** ~160 horas-persona. **Objetivo:** Construir el chat funcional end-to-end sin RAG.

Entregables:

- Backend FastAPI con healthcheck + endpoint `/api/v1/chat/messages`.
- LLMProvider abstracto funcionando con Gemini.
- Plugin Moodle con External Function `local_nexusai_chat_send`.
- Bundle React básico con MessageBubble + ChatInput + estado.
- HMAC SHA-256 funcionando entre PHP y Python.

7.3.3 Sprint 2 — RAG y carga de material (7 - 20 May 2026)

Duración: 2 semanas. **Capacidad:** ~160 horas-persona. **Objetivo:** Habilitar a docentes a subir material y al chat a usar RAG.

Entregables:

- Pipeline de extracción + chunking + embeddings (pdfplumber + tiktoken + Gemini).
- Tablas documents y chunks con migración Alembic.
- Endpoints `/api/v1/documents` (upload, list, status, delete).
- Función `retrieve_context()` con pgvector + HNSW.
- Integración RAG completa en `/api/v1/chat/messages`.
- Vista docente (`documents.php`) con drag & drop y polling.

7.3.4 Sprint 3 — Calidad y métricas (21 - 27 May 2026)

Duración: 1 semana. **Capacidad:** ~80 horas-persona. **Objetivo:** Hacer el sistema robusto para producción.

Entregables:

- BACK-11: retry con backoff en LLM/embeddings.
- BACK-12: persistencia de token counts en messages.
- BACK-13: validación de material indexado en system prompt.
- BACK-14: rate limiting + logging JSON estructurado.
- Migración 003 (token counts).
- CI/CD inicial en GitHub Actions.

7.3.5 Sprint 4 — MVP completo (28 May - 1 Jun 2026)

Duración: 5 días (sprint cerrado). **Capacidad:** ~100 horas-persona. **Objetivo:** Cerrar el MVP con 7 features finales que demuestran el alcance completo de NexusAI para la defensa.

Entregables:

- Feature A — Buscador semántico (endpoint + pestaña).
- Feature B — Chat multi-curso (toggle `□ / □`).
- Feature C — Streaming SSE (Server-Sent Events end-to-end).
- Feature D — Citas clickeables con preview del fragmento.

- Feature E — Historial de conversaciones.
- Feature F — Quiz generator con `response_format=json_object`.
- Feature G — Detección automática de gaps del docente.
- 6 issues GitHub cerradas con commit linkeado (#253-#258).
- GitHub Release v0.8.0-mvp con ZIP del plugin.

7.3.6 Documentación MVP (2 - 15 Jun 2026)

Estado: en curso al momento de redacción de este documento. **Objetivo:** Compilar la entrega final de 80+ páginas para Proyecto Integrador.

Entregables:

- Este documento (`entrega-final.pdf`).
- Presentación de defensa (15 minutos).
- Pendrive con código fuente + documentación + acceso al sistema demo.

7.4 Velocidad real vs estimada

Sprint	SP comprometidos	SP completados	Velocity	Carry-over
Sprint 0 (Setup)	60	60	100%	0
Sprint 1	50	45	90%	5
Sprint 2	60	55	92%	5
Sprint 3	25	25	100%	0
Sprint 4 (MVP)	70	70	100%	0
Total MVP	265	255	96%	10

Los 10 story points de carry-over fueron principalmente tareas de documentación menores y refactors que se relegaron sin impacto en funcionalidad. La velocidad sostenida del 96% indica una estimación relativamente precisa.

Chapter 8

Estimaciones

8.1 Metodología

El equipo estimó cada historia del backlog usando **planning poker** con escala Fibonacci (1, 2, 3, 5, 8 story points). La escala se interpreta de la siguiente manera:

SP	Interpretación
1	Trivial. Menos de 1 hora-persona. Ejemplo: cambiar un texto, agregar una validación simple.
2	Pequeño. 1-3 horas-persona. Ejemplo: agregar un endpoint trivial sin lógica nueva.
3	Medio. 3-8 horas-persona. Ejemplo: implementar un componente React con estado y API.
5	Grande. 1-2 días-persona. Ejemplo: implementar una External Function PHP nueva con HMAC.
8	Muy grande. 2-4 días-persona. Implica refactor o investigación previa. Ejemplo: pipeline RAG completo.

Cualquier historia que se estimara con más de 8 SP se dividía en sub-historias para mantener visibilidad y predictibilidad.

8.2 Story points por épica

Épica	SP totales	% del MVP
Investigación inicial	38	13%
Setup del entorno	22	8%

Épica	SP totales	% del MVP
Backend Python (FastAPI + RAG)	85	29%
Plugin Moodle (PHP)	40	14%
Frontend React (bundle AMD)	26	9%
Features Sprint 4 (A-G)	70	24%
Gestión Scrum (sprint planning, reviews, retros, dailies)	10	3%
Total MVP	291	100%

8.3 Story points por sprint

Sprint	SP planificados	SP completados	Velocity
Sprint 0 — Setup e Investigación	60	60	100%
Sprint 1 — Core chat	50	45	90%
Sprint 2 — RAG y carga de material	60	55	92%
Sprint 3 — Calidad y métricas	25	25	100%
Sprint 4 — MVP completo	70	70	100%
Total MVP	265	255	96%

8.4 Conversión SP $\square$ horas-persona

La conversión teórica usada para estimación de costos (capítulo 9) y para asegurar viabilidad del cronograma:

$$\text{SP} \times 4 \text{ horas/persona} = \text{tiempo estimado}$$

Esta conversión es aproximada — un SP medio (3) representa aproximadamente media jornada de trabajo de una persona del equipo. La métrica se calibró en el Sprint 1 cuando se midió el tiempo real dedicado vs los SP completados.

Aplicando la conversión al total de 255 SP completados:

$$255 \times 4 = 1020 \text{ horas-persona}$$

Distribuidos entre 2 personas $\square$ **~510 horas por integrante** a lo largo de las ~10 semanas de desarrollo activo del MVP. Equivale a ~50 horas-persona-semana, lo que es razonable para un proyecto académico de tiempo parcial (el equipo combina el proyecto con otras materias y, en algunos casos, con trabajo profesional part-time).

8.5 Estimaciones por integrante

Distribución aproximada de SP por persona del equipo durante el desarrollo del MVP:

Integrante	SP completados	Áreas principales
Santiago Tricherri	~140	PM, backend FastAPI, integración LLM, deploy, HMAC, multi-provider
Delfina Salinas	~115	SM, frontend React, prompt engineering, retrieval, gaps detection, documentación

La distribución no es estrictamente paritaria por rol: cada integrante tomó más peso en su área de especialización (Santiago en backend / infra, Delfina en frontend / UX / RAG / prompt engineering), pero ambos colaboraron en pair programming en componentes críticos (HMAC, pipeline RAG, features de Sprint 4).

8.6 Carry-over y deuda técnica

Al cierre del MVP quedaron **10 story points sin completar** que se relegaron como deuda técnica:

Item	SP	Motivo del carry-over
Tests automatizados del frontend (Vitest + RTL)	5	Tiempo limitado, validación manual cubre el MVP
Tests Behat del plugin PHP	3	Configurar Behat en CI requiere setup adicional
Refactor de naming en componentes React legacy	2	Cosmético, no afecta funcionalidad

Estos ítems están priorizados para post-MVP en el primer ciclo de estabilización (junio-julio 2026).

8.7 Comparación con estimación inicial

La estimación inicial (presentada en el Anteproyecto de abril 2026) proyectaba **240 SP para el MVP**. La realidad fue **265 SP planificados + 25 SP de re-estimación durante el proyecto** (Features A-G del Sprint 4 no estaban completamente desglosadas al inicio).

Métrica	Estimación inicial	Real
SP del MVP	240	291 (255 completados + 36 re-estimados)
Sprints planificados	4	5 (con Sprint 0 de investigación más largo)
Duración total	8 semanas	10 semanas
Equipo	3 personas	2 personas (50% menos staff que lo planificado)

La reducción del equipo de 3 a 2 personas a mitad del proyecto fue el mayor desvío del plan original. Se mitigó priorizando aún más el alcance y aceptando la deuda técnica documentada arriba. El MVP fue entregado a tiempo el 1 de junio de 2026, en línea con el cronograma original.

Chapter 9

Costos

9.1 Metodología

El análisis de costos se divide en cuatro categorías:

1. **Infraestructura** — hosting de backend, base de datos, cache, dominio.
2. **Servicios externos** — LLM y embeddings (Gemini en MVP, OpenAI en producción).
3. **Personal** — horas-persona del equipo valuadas a tarifa de mercado junior/semisenior en Argentina, 2026.
4. **Costos académicos** — herramientas educativas (Student Pack, Azure for Students) que reducen costos reales a cero.

9.2 Costos de infraestructura

9.2.1 Escenario MVP (estado actual, junio 2026)

Servicio	Plan	Costo mensual (USD)	Costo anual (USD)	Estado
Railway (backend FastAPI)	Free tier (\$5 USD/mes crédito)	0	0	☐ activo
Railway PostgreSQL + pgvector	Free tier (mismo proyecto)	0	0	☐ activo
Railway Redis	Free tier (mismo proyecto)	0	0	☐ activo
GitHub Free	público	0	0	☐ activo
Subtotal MVP		\$0	\$0	

9.2.2 Escenario producción (post-MVP, 500 alumnos)

Servicio	Plan	Costo mensual (USD)	Costo anual (USD)
VPS para backend (Hetzner CX22 o DO 2GB)	dedicado	6	72
PostgreSQL gestionado (DO Managed DB) o self-hosted	starter	0-15	0-180
Redis	self-hosted en mismo VPS	0	0
Dominio (.com o .ar)	—	~1	12-15
TLS / SSL	Let's Encrypt	0	0
Backup de DB	DO automated	incluido	incluido
Subtotal producción		~7-22	~84-267

9.3 Costos de servicios de IA

9.3.1 MVP con Gemini (tier gratuito)

Componente	Modelo	Costo
Chat completions	gemini-2.5-flash	\$0 (tier gratuito Google AI Studio)
Embeddings	gemini-embedding-001 (Matryoshka, 768 dim)	\$0 (tier gratuito)
Total MVP		\$0

Limitaciones del tier gratuito: 15 RPM (requests per minute), 1M TPM (tokens per minute), 1.500 RPD (requests per day) por proyecto. Suficiente para desarrollo + demo de tesis con docenas de usuarios.

9.3.2 Producción con OpenAI

Estimación basada en una facultad de 500 alumnos activos, ~10 consultas por alumno por semana, contexto de chat con 1.500 tokens de prompt + 400 de respuesta promedio.

Componente	Modelo	Costo unitario	Estimación mensual
Chat input	gpt-4o-mini	\$0.15 / M tokens	~\$50 (3×10^8 tokens)
Chat output	gpt-4o-mini	\$0.60 / M tokens	~\$48 (8×10^7 tokens)
Embeddings	text-embedding-3-small	\$0.02 / M tokens	~\$1
Total producción			~\$100/mes

Equivalente a **\$0.20 por alumno por mes**.

9.4 Costos de personal (valoración teórica)

El equipo está compuesto por 2 estudiantes de Ingeniería en Sistemas (UCC). La valoración se hace a tarifa de mercado junior con conocimientos específicos del stack (Python + React + Moodle), tomando como referencia rangos de salarios IT en Córdoba/Argentina para junio 2026.

Tarifa hora estimada: USD 12 / hora ($\approx$ ARS equivalente al cambio del momento, valuación junior con stack específico).

Sprint	Duración	Horas por persona	Horas totales	Costo (USD)
Sprint 0 — Setup e investigación	8 semanas	60	120	1.440
Sprint 1 — Core chat	6 semanas	80	160	1.920
Sprint 2 — RAG	6 semanas	80	160	1.920
Sprint 3 — Calidad	6 semanas	60	120	1.440
Sprint 4 — MVP completo	4 semanas	100	200	2.400
Documentación final	6 semanas	40	80	960
Total		420 horas	840 horas	~10.080 USD

Si el proyecto fuera comercial, el costo de desarrollo del MVP rondaría los **USD 10.000** (sin contar oficina, equipo informático, software, etc.). Este número es útil para defender el valor del proyecto frente al tribunal pero **no representa un costo real para la institución** — los integrantes son estudiantes y el trabajo es académico.

9.5 Costo total proyectado

9.5.1 Para la tesis (estado actual)

Concepto	Costo
Infraestructura MVP	\$36/año
Gemini (tier gratuito)	\$0
Personal (estudiantes)	\$0 real / ~\$10.080 valuación
Total real	~\$36/año

Gracias al GitHub Student Pack (\$200 USD de crédito DigitalOcean), el hosting de Moodle público para la defensa también queda en \$0 — cubriendo los 12 meses de defensa de tesis sin costo real.

9.5.2 Para producción comercial (1 año, 500 alumnos)

Concepto	Costo anual
Infraestructura	~\$170
OpenAI API	~\$1.200
Mantenimiento (10h/mes × \$12/h)	\$1.440

Concepto	Costo anual
Total	~\$2.810

Equivalente a **\$5.62 por alumno por año** — un orden de magnitud por debajo de soluciones comerciales tipo Khan Academy (\$59/alumno/año) o plataformas de tutoría IA con modelo similar.

9.6 Análisis costo-beneficio

El costo total para una institución que adopte NexusAI es de ~\$2.800/año para 500 alumnos. El beneficio principal es **asistencia académica disponible 24/7 sin contratar tutores adicionales**. Si la institución ahorra siquiera 1 hora-tutor por alumno por año a \$20/hora, el ahorro es \$10.000/año — un retorno de ~3.5x sobre el costo del sistema.

Adicionalmente, el sistema es **self-hosted**: una vez instalado, los datos académicos no salen de la infraestructura de la institución, lo que evita problemas regulatorios de privacidad que tienen los servicios SaaS de asistentes IA.

Chapter 10

Análisis de riesgos

10.1 Metodología

Cada riesgo identificado se evalúa con dos dimensiones:

- **Probabilidad** — Baja (1), Media (2), Alta (3).
- **Impacto** — Bajo (1), Medio (2), Alto (3).

La **severidad** es el producto Probabilidad × Impacto, en rango 1-9. Riesgos con severidad ≥ 6 requieren plan de mitigación activo; los de severidad 3-5 se monitorean; los de severidad < 3 se aceptan.

Los riesgos se agrupan en cuatro categorías: técnicos, de equipo, externos y legales/éticos.

10.2 Matriz de riesgos

ID	Riesgo	Categoría	Prob	Imp	Sev	Mitigación
R-01	Cambio en políticas o pricing de Gemini API	Externo	M	A	6	Multi-provider (ADR-003), switch a OpenAI con cambio de .env
R-02	Moodle 5.x rompe compatibilidad del plugin	Técnico	M	M	4	Matrix CI con 4.1-4.5, monitoreo de changelog y Hooks API

ID	Riesgo	Categoría	Prob	Imp	Sev	Mitigación
R-03	pgvector deja de escalar con muchos cursos	Técnico	B	A	3	HNSW + índice por course_id, particionado posterior si hace falta
R-04	Rotación / baja de un integrante del equipo (50%)	Equipo	M	A	6	Documentación exhaustiva, ADRs, pair programming, commits atómicos
R-05	Filtración de material académico privado por bug	Legal	B	A	3	Capabilities checks, Privacy API, HMAC server-to-server, tests de aislamiento
R-06	Backend Railway free tier insuficiente para la demo	Externo	M	M	4	Plan B: VPS Hetzner ~€4/mes. Plan C: Oracle Always Free
R-07	LLM alucina respuestas sin reconocer falta de material	Técnico	M	A	6	System prompt instruye decir “no encontré” + detección de gaps (Feature G) refuerza

ID	Riesgo	Categoría	Prob	Imp	Sev	Mitigación
R-08	Costo de OpenAI en producción excede presupuesto	Externo	B	M	2	Estimación: \$100/mes para 500 alumnos. Multi-provider permite retroceder a Gemini CI con moodle-plugin-ci, revisión previa contra checklist oficial
R-09	Plugin Moodle Directory rechaza la submission	Externo	M	B	2	Backups automáticos de Postgres en Railway, retención mínima 7 días
R-10	Pérdida de datos por falta de backups en backend	Técnico	B	A	3	Plan A: Student Pack DO. Plan B: Oracle Always Free. Demo local backup
R-11	Equipo no llega a deployar Moodle público para defensa	Cronograma	M	M	4	Build con polyfills, testing manual en navegadores principales
R-12	Compatibilidad navegador (widget no renderiza en Safari/Firefox viejo)	Técnico	B	M	2	

10.3 Plan detallado de los riesgos críticos (severidad ≥ 6)

10.3.1 R-01 — Cambio en políticas de Gemini API

Contexto: Gemini es nuestro LLM en MVP y permite costo cero. Si Google cambia el modelo gratuito, sube precios o deprecia la API OpenAI-compatible, la viabilidad del MVP se afecta.

Indicadores tempranos:

- Anuncios en <https://ai.google.dev/>.
- Mensajes de Deprecation en respuestas de la API.
- Cambios en rate limits del tier gratuito.

Mitigación:

- **Arquitectura multi-provider (ADR-003)** absorbe el cambio sin tocar código: solo se actualiza `.env` para apuntar al backend de OpenAI / Groq / Ollama.
- **Embeddings con gemini-embedding-001 (768 dim)** — la migración a OpenAI `text-embedding-3-small` (1.536 dim) requiere re-indexación. Tener el script de migración listo y testado antes del switch.
- **Monitoreo de costos** en `app/admin/usage` para detectar uso anómalo.

Responsable: Santiago.

10.3.2 R-04 — Rotación / baja de un integrante

Contexto: El equipo son 2 personas (Santiago + Delfina). Si uno se enferma o se ausenta, se pierde el 50% del staff. En un proyecto de tesis con plazo fijo, esto es un riesgo concreto.

Mitigación:

- **Documentación exhaustiva** — esta entrega de 80 páginas es la mitigación formal. Los ADRs, el README, los manuales y la doc de cada capa permiten que cualquier desarrollador retome el proyecto en ~1 semana.
- **Pair programming en componentes críticos** (HMAC, pipeline RAG, Hook API de Moodle) — al menos dos personas conocen cada parte.
- **Commits atómicos con mensajes descriptivos** — historia git navegable.
- **Issues con criterios de aceptación claros** — work-items son retomables por otro integrante sin contexto adicional.

Responsable: ambos integrantes, mutuamente.

10.3.3 R-07 — LLM alucina respuestas

Contexto: Si el LLM responde con seguridad cosas que el material no respalda, se rompe la promesa de “RAG auténtico” que vende el proyecto. Es el riesgo más reputacional.

Mitigación:

- **System prompt explícito:** el LLM recibe instrucciones de citar el archivo fuente y decir “no encontré información en el material” cuando no puede responder. Refinado en Sprint 4 para evitar el bug “follow-the-example” (el LLM copiaba el filename del ejemplo en el prompt).

- **Citas clickeables (Feature D)** muestran al alumno qué fragmento usó el LLM. Trazabilidad = confianza.
- **Detección de gaps (Feature G)** registra automáticamente cuando el retrieval no encontró nada o cuando el LLM admite que no puede responder. Esto da feedback al docente y al equipo.
- **Threshold de similaridad** en el retriever (0.3 en chat, 0.25 en buscador, 0.4 mínimo para quiz dirigido) filtra fragmentos irrelevantes.

Responsable: Delfina.

10.4 Riesgos aceptados sin plan activo

ID	Riesgo	Por qué se acepta
R-08	OpenAI excede presupuesto	El gasto se mide y se puede retroceder a Gemini si el costo supera \$100/mes
R-12	Compat navegadores viejos	Testing manual en Chrome/Safari/Firefox cubre 95% del uso esperado

Chapter 11

Métricas del proyecto

11.1 Métricas de proceso (Scrum)

11.1.1 Velocity por sprint

Sprint	SP comprometidos	SP completados	Velocity
Sprint 0 — Setup	60	60	100%
Sprint 1 — Core chat	50	45	90%
Sprint 2 — RAG	60	55	92%
Sprint 3 — Calidad	25	25	100%
Sprint 4 — MVP	70	70	100%
Promedio			96%

La velocidad mantenida cerca del 100% se debe a tres factores:

1. **Re-estimación honesta** en cada planning. Cuando una historia se ve más grande de lo esperado, se divide o se re-estima en vez de forzarla al sprint.
2. **Backlog priorizado**. Las historias de cada sprint se eligieron por valor + dependencias, no por tamaño.
3. **Cierre disciplinado**. Si una historia no se termina, vuelve al backlog explícitamente, no queda en limbo.

11.1.2 Carry-over por sprint

Story points planificados que terminaron en el siguiente sprint:

Sprint	Carry-over	% del sprint
0 □ 1	0 SP	0%
1 □ 2	5 SP	10%
2 □ 3	5 SP	8%
3 □ 4	0 SP	0%
4 □ backlog	10 SP	14% (deuda técnica documentada)

11.1.3 Burn-down ideal vs real

xychart-beta

```
title "Burn-down del MVP (SP completados acumulados)"
x-axis "Sprint" [Sprint 0, Sprint 1, Sprint 2, Sprint 3, Sprint 4]
y-axis "SP acumulados" 0 --> 280
line "Ideal" [55, 110, 165, 220, 265]
line "Real" [60, 105, 160, 185, 255]
```

El equipo terminó el MVP con **255 SP completados sobre 265 planificados** — una ejecución del 96%. Los 10 SP de carry-over son la deuda técnica documentada en el capítulo 8.

11.2 Métricas de desarrollo

Métrica	Valor	Fuente
Total commits al repo	~250	git log
Total issues cerradas	70+	GitHub issues
Pull Requests mergeados	~50	GitHub PRs
Líneas de código backend Python	~3.500	cloc services/api/app
Líneas de código plugin PHP	~2.500	cloc plu- gin/local/nexusai/classes
Líneas de código React (sin bundle compilado)	~3.200	cloc plu- gin/local/nexusai/react/src
Migraciones de DB	5 (001 a 004)	services/api/migrations/versions/
ADRs documentadas	6	docs/adr/
Diagramas Mermaid	5	docs/diagrams/
Documentos de investigación	47	investigacion/
Tests automatizados backend	37	services/api/tests/
Cobertura de tests backend	~80%	pytest --cov
Workflows CI/CD	4	.github/workflows/

11.3 Métricas del producto en runtime

Mediciones tomadas sobre la instancia del backend deployado en Railway durante el desarrollo del Sprint 4:

Métrica	Valor objetivo (RNF)	Valor medido
Latencia al primer token (streaming)	< 2 s	~0.8 - 1.2 s
Latencia respuesta completa (chat sync)	< 8 s	~3 - 6 s
Latencia del buscador (retrieval puro)	< 1 s	~0.3 - 0.8 s

Métrica	Valor objetivo (RNF)	Valor medido
Tiempo de indexación PDF 50 páginas	< 60 s	~30 - 50 s
Hit rate del LLM (1ra vez)	—	100% (sin cache MVP)
Tasa de errores 5xx en producción	< 1%	< 0.5% (medido sobre /health checks)

Mediciones de uso de la demo durante los smoke tests E2E:

Métrica	Valor de prueba
Documentos indexados (cursos de demo)	8
Chunks vectorizados	~520
Tamaño promedio de chunk	512 tokens
Embeddings dimensiones	768 (Gemini Matryoshka)
Sesiones de chat creadas	~30
Quizzes generados	~12
Gaps registrados	~25

11.4 Calidad del LLM (golden set)

Para evaluar la calidad de las respuestas del asistente, el equipo mantiene un **golden set** de preguntas con respuestas esperadas, en `services/api/tests/golden_set.md`. Cada release se valida manualmente contra ese set:

Métrica	Resultado en MVP
Preguntas con respuesta correcta + citación adecuada	8/10
Preguntas donde el LLM admitió no poder responder (correcto)	2/2
Falsos positivos (LLM responde con confianza algo incorrecto)	0
Falsos negativos (LLM no responde teniendo material disponible)	0

El golden set se expandirá en post-MVP a 50+ preguntas con métricas formales (BLEU, exact match) corridas automatizadamente en CI.

11.5 Métricas de costos

Concepto	Valor MVP	Proyección producción
Infraestructura	~\$3/mes	~\$15/mes
LLM (Gemini gratuito vs OpenAI prod)	\$0	~\$100/mes (500 alumnos)
Total operativo	~\$3/mes	~\$115/mes

Detalle completo de costos en el capítulo 9.

11.6 Métricas de gestión

Métrica	Valor
Sprint Planning realizados	5 (uno por sprint)
Sprint Reviews + Retros realizados	5
Dailies asíncronas en GitHub Projects	~70 (~7 por integrante por sprint)
ADRs aceptadas	6
Pull Requests con review (no merge directo)	~85%

La política del equipo fue: ningún merge a main sin review del otro integrante, salvo bugfixes triviales o cambios de documentación contenidos. Esto sumó tiempo de coordinación pero atrapó varios bugs antes de llegar a producción.

Chapter 12

Sprint 0 — Setup e Investigación

12.1 Objetivo del sprint

TODO — Investigar tecnologías clave (RAG, pgvector, Moodle plugins) y montar el entorno de desarrollo.

12.2 Duración

TODO — desde / hasta.

12.3 Tareas completadas

- ☒ Relevamiento de stakeholders (investigacion/09-relevamiento/)
- ☒ Investigación técnica:
 - ☒ Moodle architecture y plugin development (investigacion/01-moodle/)
 - ☒ RAG y embeddings (investigacion/02-rag/)
 - ☒ OpenAI/Gemini APIs (investigacion/03-openai/)
 - ☒ PHP en Moodle (investigacion/04-php-moodle/)
 - ☒ FastAPI backend (investigacion/05-backend-fastapi/)
 - ☒ Vector databases (investigacion/06-vector-databases/)
 - ☒ UI/UX para chat académico (investigacion/07-ui-ux/)
 - ☒ Seguridad y arquitectura (investigacion/08-seguridad-arquitectura/)
 - ☒ Flujo RAG end-to-end (investigacion/09-flujo-rag/)
- ☒ Setup del entorno de desarrollo (investigacion/10-setup-entorno/)
- ☒ Repo y herramientas (Git, GitHub, Docker, Pandoc)

12.4 Entregables

- 47 documentos de investigación
- Repositorio inicializado
- ADRs base (001, 002, 003)
- Decisión de stack final

12.5 Retrospectiva

TODO — Qué salió bien, qué no, qué aprendimos.

Chapter 13

Sprint 1 — Core chat

13.1 Objetivo del sprint

TODO — Implementar la pieza central: chat alumno-asistente con backend FastAPI y plugin Moodle mínimo viable.

13.2 Duración

TODO

13.3 Tareas completadas

- ☐ Backend: endpoint `/api/v1/chat/messages` con HMAC
- ☐ Backend: LLMProvider abstracto (Gemini compat OpenAI)
- ☐ Plugin Moodle: estructura `local_nexusai` + hooks
- ☐ Plugin: External Function `local_nexusai_chat_send`
- ☐ React: `ChatApp.jsx` + `MessageBubble` + `ChatInput`

13.4 Entregables

- Plugin v0.1.x instalable
- Chat funcional sin RAG (solo conversación)
- Tests de HMAC pasando

13.5 Retrospectiva

TODO

Chapter 14

Sprint 2 — RAG y carga de material

14.1 Objetivo

TODO — Habilitar a docentes a subir material y al chat a usar RAG.

14.2 Duración

TODO

14.3 Tareas completadas

- ☐ Backend: pipeline de procesamiento de PDFs (extracción + chunking + embeddings)
- ☐ Backend: `/api/v1/documents` (upload, list, status, delete)
- ☐ Backend: `retrieve_context()` con pgvector + HNSW
- ☐ Backend: integración RAG en `/chat/messages`
- ☐ Plugin: External Functions de gestión de documentos
- ☐ Plugin: página `documents.php` con drag & drop

14.4 Entregables

- Plugin v0.3.0 con vista docente
- Backend con RAG funcional
- Migración 002 (HNSW)

14.5 Retrospectiva

TODO

Chapter 15

Sprint 3 — Calidad y métricas

15.1 Objetivo

TODO — Hacer el sistema robusto: retries, logging estructurado, rate limiting, métricas de tokens.

15.2 Duración

TODO

15.3 Tareas completadas

- ☐ BACK-11: retry con backoff en LLM/embeddings
- ☐ BACK-12: persistencia de token counts
- ☐ BACK-13: validación de material indexado en system prompt
- ☐ BACK-14: rate limiting por user_id + logging JSON estructurado
- ☐ Migración 003 (token counts)
- ☐ CI/CD inicial en GitHub Actions

15.4 Entregables

- Plugin v0.4.0 estable
- Pipeline CI/CD funcional
- Tests > 60% cobertura

15.5 Retrospectiva

TODO

Chapter 16

Sprint 4 — MVP completo

16.1 Objetivo

Cerrar el MVP con 7 features que demuestran el alcance completo de NexusAI para la defensa.

16.2 Duración

mayo 2026 — junio 2026.

16.3 Features entregadas

16.3.1 Feature A — Buscador semántico

Versión: 0.4.0 · **Commit:** ef44ad6 · **Issue:** #253

Endpoint y UI de retrieval puro sobre pgvector. Devuelve fragmentos relevantes del material del curso a una consulta del alumno, sin pasar por el LLM.

16.3.2 Feature B — Chat multi-curso

Versión: 0.4.0 · **Commit:** ef44ad6 · **Issue:** #254

Toggle ☐ / ☒ que permite al alumno consultar material de TODOS sus cursos a la vez, no solo el actual. El LLM cita la materia de cada fragmento.

16.3.3 Feature C — Streaming SSE

Versión: 0.5.0 · **Commit:** 6a15cd7 · **Issue:** #255

Respuesta token-por-token con Server-Sent Events. Primer token en ~1s vs ~8s del modo sync. Mantiene Hybrid PHP Proxy (ADR-001) — HMAC server-to-server.

16.3.4 Feature D — Citas clickeables con preview

Versión: 0.6.0 · **Commit:** 27ef377 · **Issue:** #256

Las pills de “Fuentes.” son botones que expanden el fragmento exacto usado por el LLM, con score de similaridad. Cierra el loop visual del RAG.

16.3.5 Feature E — Historial de conversaciones

Versión: 0.7.0 · **Commit:** 9b426b9 · **Issue:** #257

Dropdown ☐ con sesiones previas del alumno, ordenadas por última actividad. Click ☐ carga mensajes y permite continuar la conversación.

16.3.6 Feature F — Quiz generator

Versión: 0.8.0 · **Commit:** 5f9f6c5 · **Issue:** #258

Pestaña “Quiz” que genera preguntas de opción múltiple sobre el material. 4 opciones, feedback inmediato con explicación + archivo fuente, score final.

16.3.7 Feature G — Detección de gaps del docente

Versión: 0.9.0 · **Commit:** TODO

Sistema registra automáticamente preguntas de alumnos que el material no pudo responder (chunks=0 o max_sim<0.4 o LLM dice “no encontré”). El docente ve un reporte de gaps en NexusAI · Materials. Feedback loop pedagógico.

16.4 Entregables del sprint

- Plugin v0.9.x con las 7 features
- 6 issues cerradas con commit linkeado (#253-#258)
- GitHub Release v0.8.0-mvp con ZIP descargable
- Backend deployado en Railway (autodeploy via GitHub Actions)
- README con instrucciones de instalación para cualquier Moodle

16.5 Retrospectiva

TODO — Qué salió bien (streaming, citas), qué costó (LLM “follow-the-example” en prompts, gaps con señal solo del retrieval), qué cambiaríamos.

Chapter 17

Arquitectura técnica

17.1 Visión general

NexusAI es un **plugin Moodle con asistente IA** que combina tres capas:

1. **Plugin tipo local** dentro de Moodle (PHP) que inyecta un widget en todas las páginas de curso y expone un endpoint AJAX seguro.
2. **Frontend React** compilado como módulo AMD, embebido en el plugin.
3. **Backend Python (FastAPI)** que orquesta el pipeline RAG: recupera fragmentos relevantes del material del curso desde PostgreSQL/pgvector y genera respuestas con el LLM activo (Gemini Flash en MVP, GPT-4o-mini en producción).

La pieza diferencial es el **RAG auténtico**: el material que el docente sube a Moodle se indexa automáticamente, y la IA responde con citas a la fuente. Si la pregunta no se puede responder con el material disponible, el sistema lo admite explícitamente — no inventa.

17.1.1 Principios rectores

- **Una sola base de datos.** PostgreSQL con la extensión pgvector cubre datos relacionales y vectores. No hay ChromaDB ni base vectorial separada.
- **Agnóstico de proveedor LLM.** El backend abstrae el proveedor detrás de las clases LLM-Provider y EmbeddingProvider. Cambiar de Gemini a OpenAI es solo modificar variables de entorno.

17.2 Diagrama de componentes

flowchart TB

```
subgraph BROWSER["Navegador del alumno"]
    REACT["React 18<br/>(bundle AMD)"]
end
```

```
subgraph MOODLE["Moodle (servidor universidad)"]
    PHP["Plugin local_nexusai<br/>(PHP)"]
    MFILES[("mdl_files<br/>+ tablas locales propias")]
end
```



```

end

subgraph BACKEND["Backend NexusAI"]
  FASTAPI["FastAPI<br/>(monolito modular)"]
  PG["PostgreSQL + pgvector<br/>documents · chunks<br/>chat_sessions · messages"]
  REDIS["Redis<br/>HMAC nonces + cache"]
end

subgraph EXTERNAL["Externo (configurable)"]
  LLM["LLMProvider<br/>Gemini Flash (MVP)<br/>GPT-4o-mini (prod)"]
  EMB["EmbeddingProvider<br/>Gemini Embedding (MVP)<br/>text-embedding-3-small ("]
end

REACT -->|core/ajax + sesskey| PHP
PHP -->|cURL + HMAC| FASTAPI
PHP <--> MFILES
FASTAPI <-->|SQL + vector| PG
FASTAPI <--> REDIS
FASTAPI -->|API key| LLM
FASTAPI -->|API key| EMB

```

17.3 Patrón Hybrid PHP Proxy (ADR-001)

El navegador del alumno **nunca habla directo con el backend Python**. Toda llamada del frontend pasa por el plugin Moodle, que actúa como proxy autenticado con HMAC server-to-server. Esto resuelve tres problemas a la vez:

- **La API key del LLM nunca llega al navegador.** Vive como variable de entorno en el backend Python. No hay forma de filtrarla desde el cliente.
- **CSRF queda cubierto** por la sesskey nativa de Moodle, validada por core/ajax.
- **CORS no es un problema** porque la única request cross-origin que hay es server-to-server (PHP ↔ Python) con cURL.

El costo es que cada feature nueva requiere implementar el endpoint en 3 capas (Python ↔ External Function PHP ↔ cliente React), pero la separación mantiene la auditabilidad de cualquier institución que instale el plugin: el HMAC, el secret y los logs viven en su infraestructura.

17.4 Flujo de una consulta del alumno

```

sequenceDiagram
    autonumber
    actor A as Alumno
    participant R as React (browser)
    participant P as Plugin PHP (Moodle)
    participant F as FastAPI (backend)
    participant PG as PostgreSQL/pgvector
    participant E as EmbeddingProvider

```

participant L as LLMProvider

```

A-->R: Escribe pregunta
R-->P: core/ajax → local_nexusai_chat_send
P-->P: validate_context + require_capability
P-->P: backend_client::send_message()<br/>computa HMAC(secret, ts + nonce + body)
P-->F: POST /api/v1/chat/messages<br/>+ Authorization + X-Timestamp + X-Nonce + X-
F-->F: verify_hmac (3 capas: API key + firma + nonce Redis)
F-->PG: get_or_create chat_session<br/>+ INSERT user message
F-->E: embed(question)
E-->F: vector 768 dim
F-->PG: SELECT chunks ORDER BY embedding <=> $1<br/>JOIN documents WHERE course_id
PG-->F: top-5 chunks + metadata
F-->F: build prompt (system + historial + contexto + pregunta)
F-->L: chat_completion(messages)
L-->F: respuesta del LLM
F-->PG: INSERT assistant message
F-->P: JSON {session_id, answer, messages[]}
P-->R: response al cliente core/ajax
R-->A: respuesta aparece en panel

```

Latencia con respuesta no-streaming: 3–6 s end-to-end.

Latencia en modo streaming SSE (Sprint 3 — Feature C): ~1 s al primer token, 3–6 s respuesta completa.

17.5 Pipeline RAG — indexación offline

Cuando un docente sube material nuevo, el pipeline corre asíncrono mediante BackgroundTasks de FastAPI:

flowchart LR

```

PDF[PDFs/DOCX/TXT<br/>en mdl_files] --> EXT[extractor.py<br/>pdfplumber]
EXT --> CHUNK[chunker.py<br/>tiktoken cl100k_base<br/>512 tokens / 64 overlap]
CHUNK --> EMB[EmbeddingProvider<br/>gemini-embedding-001<br/>Matryoshka 768 dim]
EMB --> STORE[(pgvector<br/>INSERT INTO chunks)]
EMB --> STATUS[document.status = 'indexed']

```

Detalles clave:

- **Extracción sin OCR.** El extractor rechaza PDFs escaneados con un error explícito. Solo se aceptan PDFs con texto extraíble.
- **Chunking de 512 tokens con overlap de 64** (12.5%). El solapamiento preserva contexto en los bordes de los chunks y mejora la calidad del retrieval. El conteo de tokens usa cl100k_base, compatible tanto con Gemini como con OpenAI.
- **Persistencia transaccional.** Si la extracción o el embedding falla, el documento queda marcado con status='error' y error_message poblado, sin chunks parciales en la base.
- **Estado del documento.** Ciclo pending → indexing → indexed | error, persistido en documents.status y consultado por el frontend con polling cada 3 segundos.

Costo de indexación: \$0 con Gemini en el tier gratuito del MVP. Aproximadamente \$0.10 por cada 10.000 chunks con OpenAI en producción.

17.6 Estados del proyecto y trayectoria post-MVP

El backend es un monolito modular: una sola aplicación FastAPI con módulos bien separados (chat, documents, search, quiz, gaps). Esta decisión está formalizada en ADR-001 y permite extraer servicios cuando el dolor lo justifique:

```
flowchart LR
    subgraph MVP["MVP (1 servicio)"]
        API["FastAPI<br/>chat + documents + analytics"]
    end

    subgraph V2["Post-MVP (cuando aparezca el dolor)"]
        APIV2["nexus-api<br/>chat + queries"]
        IDXV2["nexus-indexer<br/>worker async"]
        ANAV2["nexus-analytics<br/>queries agregadas"]
    end

    MVP --> V2
```

Cuándo extraer	Qué servicio	Por qué
Sprint 5-6 (post-MVP)	nexus-indexer como worker	Indexar 200 PDFs no debe bloquear el API
Inicio de Épica 04 (analytics docente)	nexus-analytics con DB propia	Aislar queries pesadas del chat
Si pgvector no escala (>10M vectores)	Migrar a Qdrant/Weaviate	Diseñado para single-institution, lejos de ese límite hoy

Chapter 18

Stack tecnológico

18.1 Tabla de componentes

Capa	Tecnología	Versión / detalle
Frontend	React + Webpack	React 18.3, Webpack 5, bundle único AMD
Plugin Moodle	PHP	8.0+
Compatibilidad Moodle	Moodle	4.1 LTS – 4.5 (Hook API nuevo en 4.4+, callback legacy en 4.1-4.3)
Backend IA	FastAPI + Uvicorn	FastAPI 0.115, Python 3.11
ORM + migraciones	SQLAlchemy 2.0 async + Alembic	engine asyncpg, modelos en <code>app/db/models.py</code>
Base de datos + vectores	PostgreSQL + pgvector	PG 16, pgvector 0.3.5 con índice HNSW
LLM (MVP)	Gemini 2.5 Flash	Tier gratuito vía SDK
LLM (producción)	GPT-4o-mini	OpenAI-compatible
Embeddings (MVP)	<code>gemini-embedding-001</code> (Matryoshka)	API OpenAI
Embeddings (producción)	<code>text-embedding-3-small</code> (OpenAI)	768 dim
Cache + nonces HMAC	Redis	1.536 dim — requiere re-indexación
Containers	Docker Compose	7 alpine
CI/CD	GitHub Actions	profiles dev, full, tools
		backend-ci, frontend-ci, moodle-ci, deploy a Railway

18.2 Justificación de las decisiones clave

Cada decisión está documentada formalmente como un ADR (Architectural Decision Record) en `docs/adr/`. Resumen:

18.2.1 Monolito modular sobre microservicios

El equipo es de 2 personas con un deadline corto. Un monolito modular cubre todas las necesidades del MVP sin la complejidad operativa de microservicios (orquestración, observabilidad distribuida, contratos entre servicios). Los módulos del backend están bien separados — chat, documents, search, quiz, gaps — lo que permite extraer servicios cuando aparezca el dolor real (típicamente: un worker de indexación async cuando los uploads bloqueen el API).

□ Detalle en ADR-001.

18.2.2 pgvector sobre ChromaDB

ChromaDB sería un sistema separado de PostgreSQL, lo que duplica la operación (dos bases, dos backups, dos puntos de falla) y bloquea queries que combinen SQL+vector en una transacción. pgvector cubre la escala del proyecto (single-institution, <10M vectores) con índice HNSW de aproximación, y permite hacer joins entre chunks y documents sin sacarlos de la DB.

□ Detalle en ADR-002.

18.2.3 Arquitectura multi-provider LLM

El backend abstrae el LLM y el provider de embeddings detrás de las clases `LLMProvider` y `EmbeddingProvider`. Esto permite:

- Iterar gratis con Gemini 2.5 Flash en MVP.
- Cambiar a OpenAI GPT-4o-mini en producción modificando solo variables de entorno.
- Probar con Ollama local para desarrollo offline sin tocar código de aplicación.

□ Detalle en ADR-003.

18.2.4 Gemini 2.5 Flash en MVP, OpenAI en producción

Gemini ofrece un tier gratuito generoso que permite desarrollar e iterar sin costos. Para producción, GPT-4o-mini de OpenAI da mejor calidad de respuesta a un costo bajo (~\$0.15 por millón de tokens input). El switch entre proveedores es transparente gracias a la abstracción multi-provider.

□ Detalle en ADR-004.

18.2.5 HMAC SHA-256 en 3 capas

La comunicación entre el plugin Moodle (PHP) y el backend (Python) usa HMAC SHA-256 con tres capas de seguridad:

1. **Bearer API key** identifica a la instancia de Moodle.
2. **Firma HMAC** sobre `timestamp || nonce || body` evita tampering.
3. **Nonce store en Redis** con TTL de 5 minutos evita replay attacks.

Los secretos viven solo en la configuración del plugin (Moodle admin) y en variables de entorno del backend. El navegador nunca los ve.

□ Detalle en ADR-005.

18.2.6 Privacy API con `null_provider` en MVP

El plugin no persiste datos personales en Moodle (historial de chat, sesiones, documentos indexados — todo eso vive en el backend Python externo). En consecuencia, la Privacy API del plugin declara un `null_provider`. Si más adelante se agregan tablas locales en Moodle (audit log de uso, cache de respuestas), la implementación migra a `metadata\provider`.

□ Detalle en ADR-006.

18.3 Tecnologías evaluadas y descartadas

Alternativa	Por qué no
Microservicios desde el inicio	Equipo de 2 personas, deadline corto, sin tracción de usuarios todavía
ChromaDB	Sistema separado de PostgreSQL — duplica operación y bloquea queries SQL+vector
Pinecone (managed)	\$70+/mes mínimo, vendor lock-in. Innecesario para single-institution
Qdrant / Weaviate	Justificados a partir de 10-50M vectores, fuera del rango
Fine-tuning del LLM en lugar de RAG	Costoso, requiere re-train por curso, menos flexible que retrieval
Vite en lugar de Webpack	Vite no tiene buen soporte para output AMD que necesita Moodle
LLM local (Llama, Mistral)	Fuera de alcance MVP; la abstracción permite agregarlo después sin cambios estructurales
Subsistema IA nativo de Moodle 4.5	Solo soporta <code>generate_text</code> , <code>generate_image</code> , <code>summarise_text</code> . Sin acción “chat” nativa
API key OpenAI fija (sin abstracción)	Incompatible con MVP gratuito (Gemini) — la abstracción multi-provider es estructural

Chapter 19

Modelo de datos

19.1 Visión general

NexusAI usa **una sola base de datos PostgreSQL** con la extensión pgvector activada. Todos los datos del proyecto — incluyendo embeddings vectoriales — viven en la misma instancia, lo que permite hacer joins SQL entre tablas relacionales y la columna vectorial dentro de una transacción.

Las tablas se dividen en dos grupos según quién las gestiona:

- **Tablas del backend Python** (documents, chunks, chat_sessions, messages, unanswered_questions) — gestionadas por Alembic, incluyen el tipo vector(N) de pgvector.
- **Tablas del plugin Moodle** (local_nexusai_*) — gestionadas por XMLDB de Moodle (plugin/local/nexusai/db/install.xml). Hoy el MVP usa null_provider de la Privacy API y no persiste datos personales en estas tablas (ADR-006).

19.2 Diagrama entidad-relación

erDiagram

```
DOCUMENTS ||--o{ CHUNKS : "compone"
CHAT_SESSIONS ||--o{ MESSAGES : "contiene"

DOCUMENTS {
    uuid id PK
    int course_id "ID del curso Moodle"
    int uploader_id "ID del docente Moodle"
    string(255) filename
    string(100) mime_type
    string(20) status "pending | indexing | indexed | error"
    text error_message NULL
    string(64) file_hash "SHA-256 para dedup"
    timestampz created_at
    timestampz updated_at
}
```

```
CHUNKS {
    uuid id PK
    uuid document_id FK
    text content
    int chunk_index
    int token_count NULL
    vector embedding "Vector(768) MVP / Vector(1536) prod"
    timestampz created_at
}

CHAT_SESSIONS {
    uuid id PK
    int user_id "ID del alumno Moodle"
    int course_id "ID del curso Moodle (0 = multi-curso)"
    timestampz created_at
    timestampz updated_at
}

MESSAGES {
    uuid id PK
    uuid session_id FK
    string(20) role "user | assistant | system"
    text content
    int token_count_prompt NULL
    int token_count_completion NULL
    timestampz created_at
}

UNANSWERED_QUESTIONS {
    uuid id PK
    int course_id
    int user_id
    text question
    float max_similarity NULL
    int chunks_retrieved
    timestampz created_at
}
```

19.3 Tablas

19.3.1 documents

Material que el docente sube al curso para ser indexado.

Columna	Tipo	Notas
id	UUID PK	generado en el servidor con <code>uuid4()</code>
course_id	INT NOT NULL	ID del curso Moodle
uploader_id	INT NOT NULL	<code>\$USER->id</code> del docente que subió el archivo
filename	VARCHAR(255)	nombre del archivo original
mime_type	VARCHAR(100)	MVP solo acepta <code>application/pdf</code>
status	VARCHAR(20)	<code>pending</code> ☐ <code>indexing</code> ☐ <code>indexed</code> (o <code>error</code>)
error_message	TEXT NULL	poblado solo si <code>status='error'</code>
file_hash	VARCHAR(64) NULL	SHA-256 del contenido para detectar uploads duplicados (migración 004)
created_at, updated_at	TIMESTAMPTZ	server defaults

Índice único parcial: (`course_id`, `filename`, `file_hash`) WHERE `file_hash IS NOT NULL`. Garantiza que no se indexe dos veces el mismo archivo en el mismo curso, pero deja pasar documentos antiguos sin hash (compat con datos pre-migración 004).

19.3.2 chunks

Fragmentos de cada documento, ya vectorizados.

Columna	Tipo	Notas
id	UUID PK	
document_id	UUID NOT NULL FK	ON DELETE CASCADE
content	TEXT NOT NULL	texto del fragmento (~512 tokens)
chunk_index	INT NOT NULL	orden dentro del documento
token_count	INT NULL	conteo <code>cl100k_base</code>
embedding	vector(768)	dimensión Matryoshka del modelo Gemini
created_at	TIMESTAMPTZ	

Índice HNSW sobre `embedding` con distancia coseno, parámetros `m=16` y `ef_construction=200`. Permite búsqueda aproximada de vecinos más cercanos en tiempo casi constante. La migración 002 lo crea explícitamente.

19.3.3 chat_sessions

Una sesión de conversación por usuario por curso (o `course_id=0` para sesión multi-curso de Feature B).

Columna	Tipo	Notas
id	UUID PK	
user_id	INT NOT NULL	ID del alumno
course_id	INT NOT NULL	ID del curso, o 0 si es multi-curso
created_at, updated_at	TIMESTAMPTZ	

19.3.4 messages

Mensajes individuales dentro de una sesión.

Columna	Tipo	Notas
id	UUID PK	
session_id	UUID NOT NULL FK	ON DELETE CASCADE
role	VARCHAR(20)	user / assistant / system
content	TEXT NOT NULL	mensaje completo
token_count_prompt	INT NULL	tokens del prompt (solo en role='assistant')
token_count_completion	INT NULL	tokens de la respuesta (solo en role='assistant')
created_at	TIMESTAMPTZ	

Los token counts se agregaron en la migración 003 para habilitar monitoreo de costos en tiempo real.

19.3.5 unanswered_questions

Tabla del feedback loop pedagógico (Feature G). Cada vez que el sistema detecta que el material indexado no pudo responder bien a una pregunta del alumno, registra el evento.

Columna	Tipo	Notas
id	UUID PK	
course_id	INT NOT NULL	
user_id	INT NOT NULL	
question	TEXT NOT NULL	pregunta original, truncada a 2000 chars
max_similarity	FLOAT NULL	máxima similitud encontrada (NULL si chunks=0)
chunks_retrieved	INT NOT NULL	cantidad de chunks que pasaron el threshold
created_at	TIMESTAMPTZ	

Criterio de registro: si `chunks_retrieved=0` O `max_similarity<0.4` O la respuesta del LLM matchea patrones tipo “*no se encuentra en el material*”. Detalle en el capítulo 21 (Testing) y en la implementación de `app/gaps/recorder.py`.

19.4 Evolución del schema (migraciones Alembic)

Migración	Cambio	Sprint
001_initial_schema	Tablas base: documents, chunks, chat_sessions, messages	Sprint 1
002_hnsw_index	Índice HNSW en chunks.embedding con distancia coseno	Sprint 2
003_message_token_counts	Columnas token_count_prompt y token_count_completion en messages	Sprint 3
004_document_hash	Columna file_hash en documents + índice único parcial para detectar duplicados	Sprint 4
004_unanswered_questions	Tabla unanswered_questions para Feature G	Sprint 4

Nota técnica: las dos migraciones del Sprint 4 fueron desarrolladas en paralelo por integrantes distintos del equipo y quedaron numeradas igual (004_*). Al integrarse, se encadenaron explícitamente — 004_document_hash depende de 004_unanswered_questions — para linealizar el árbol de Alembic. Es un patrón conocido cuando dos personas crean migraciones desde el mismo padre.

19.5 Migración entre dimensiones de embeddings (post-MVP)

El campo chunks.embedding está hoy en vector(768) (Gemini, modelo Matryoshka). Para producción con OpenAI text-embedding-3-small (1536 dim), la migración es:

```
ALTER TABLE chunks ALTER COLUMN embedding TYPE vector(1536);
DROP INDEX chunks_embedding_idx;
CREATE INDEX chunks_embedding_idx ON chunks
    USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 200);
-- + re-indexar todos los chunks (re-procesar PDFs con el nuevo modelo)
```

HNSW no soporta cambio de dimensión in-place, por eso hay que destruir y recrear el índice.

19.6 Tablas del plugin Moodle (local_nexusai_*)

Hoy el MVP usa solo local_nexusai_placeholder (vacía, evita warnings del plugin checker de Moodle). Las siguientes tablas están planificadas para post-MVP:

- local_nexusai_usage — rate limiting por (usuario, curso, día)

- `local_nexusai_cache` — cache de respuestas LLM por hash de pregunta
- `local_nexusai_course_settings` — settings configurables por curso

Cuando se implementen, se actualiza la Privacy API del plugin migrando de `null_provider` a `metadata_provider` (ADR-006).

Chapter 20

API endpoints

El backend FastAPI expone 16 endpoints HTTP organizados en seis grupos funcionales: salud (sin auth), chat, documentos, búsqueda semántica, quiz, gaps y admin. Todos los endpoints con prefijo `/api/v1/` requieren autenticación HMAC (excepto un endpoint de smoke test).

20.1 Autenticación HMAC

Todos los endpoints (excepto `/health` y `/`) requieren cuatro headers:

Header	Valor
Authorization	Bearer <NEXUSAI_API_KEY>
X-Timestamp	Unix epoch en segundos, como string
X-Nonce	16 bytes hexadecimales aleatorios
X-Signature	hex(HMAC_SHA256(secret, timestamp nonce body))

El plugin Moodle (PHP) genera estos headers automáticamente en `backend_client::compute_signature()`. El backend (Python) los valida en `app/auth/hmac.py` con tres capas:

1. Comparación constant-time del Bearer API key.
2. Recálculo de la firma sobre el body raw recibido.
3. Verificación de que el nonce no está en Redis (anti-replay, TTL 5 minutos).

Si cualquiera de las tres falla, FastAPI corta la request con `401 Unauthorized` antes de llegar al handler.

20.2 Endpoints de salud

20.2.1 GET `/health`

Liveness probe. Sin autenticación. Devuelve estado del backend y modelo LLM activo. Usado por Docker, Kubernetes y monitoreo externo.

```
{ "status": "ok", "version": "0.9.4", "env": "production", "llm_model": "gemini-2.5-fl
```

20.2.2 GET /

Endpoint raíz. Sin autenticación. Devuelve { "message": "NexusAI API running" }.

20.3 Chat

20.3.1 POST /api/v1/chat/messages

Endpoint sync original. Recibe una pregunta, hace retrieval, llama al LLM y devuelve la respuesta completa cuando termina.

Request body:

```
{
  "question": "string (1-2000)",
  "course_id": 42,
  "user_id": 7,
  "session_id": "uuid | null",
  "course_ids": [42, 51],
  "course_names": { "42": "Cálculo I", "51": "Programación I" }
}
```

Los campos `course_ids` y `course_names` son opcionales y habilitan el modo multi-curso (Feature B).

Response:

```
{
  "session_id": "uuid",
  "answer": "respuesta completa del LLM",
  "messages": [ /* historial actualizado */ ],
  "prompt_tokens": 1456,
  "completion_tokens": 312,
  "total_tokens": 1768
}
```

20.3.2 POST /api/v1/chat/stream

Versión streaming del endpoint anterior. Devuelve `StreamingResponse` con Server-Sent Events. Implementa Feature C.

Eventos SSE emitidos (line-delimited JSON con prefijo `data:`):

Tipo	Contenido	Cuándo
meta	session_id, chunks (cantidad), sources (chunks completos con texto, similarity, course_id), course_names (si multi-curso)	Primero, una sola vez
token	content (string del chunk de texto)	N veces, una por chunk del LLM
done	prompt_tokens, completion_tokens, total_tokens	Al final
error	detail (string)	Si falla algo durante el stream

El cliente PHP forwarda estos eventos al browser sin tocarlos vía un endpoint dedicado `/local/nexusai/chat_stream.php` con `CURLOPT_WRITEFUNCTION`.

20.3.3 POST `/api/v1/chat/sessions/list`

Lista las sesiones previas del alumno. Implementa Feature E.

Body:

```
{ "user_id": 7, "course_id": 42, "limit": 20 }
```

Si `course_id` es null, devuelve sesiones de todos los cursos del usuario.

Response: array de sesiones con `id`, `course_id`, `created_at`, `updated_at`, `last_message_preview`, `message_count`.

20.3.4 POST `/api/v1/chat/sessions/messages`

Devuelve los mensajes completos de una sesión específica. Valida que la sesión pertenezca al `user_id` (ownership check a nivel app).

20.3.5 POST `/api/v1/chat/echo`

Endpoint mock para smoke test del HMAC desde el cliente PHP. No procesa nada; solo valida la firma y devuelve eco del payload.

20.4 Documentos

20.4.1 POST `/api/v1/documents`

Sube un documento para indexación RAG. El archivo viaja como base64 dentro de un JSON (no multipart) para que la firma HMAC sea predecible.

Body:

```
{
  "course_id": 42,
  "uploader_id": 7,
  "filename": "apunte-derivadas.pdf",
  "mime_type": "application/pdf",
  "content_b64": "JVBERi0xLjQK..."
}
```

Response (HTTP 202 Accepted): el documento se persiste con `status = indexing` y un `BackgroundTask` corre la indexación `async`. El frontend hace polling de `/api/v1/documents/{id}` para ver el estado.

Detección de duplicados: el backend computa SHA-256 del contenido y lo guarda en `documents.file_hash`. Si ya existe el mismo hash en el mismo curso con `status='indexed'`, devuelve el documento existente sin re-indexar (idempotente).

20.4.2 GET /api/v1/documents?course_id=N

Lista todos los documentos indexados del curso N ordenados por `created_at DESC`.

20.4.3 GET /api/v1/documents/{document_id}

Devuelve el estado actual de un documento. Usado por el frontend para polling durante la indexación.

20.4.4 DELETE /api/v1/documents/{document_id}

Borra un documento. PostgreSQL hace `ON DELETE CASCADE` sobre chunks automáticamente.

20.5 Búsqueda semántica (Feature A)

20.5.1 POST /api/v1/search

Retrieval puro sobre pgvector — devuelve fragmentos relevantes sin pasar por el LLM. Más permisivo que el retrieval del chat (`min_similarity=0.25`).

Body:

```
{ "query": "string", "course_id": 42, "user_id": 7, "top_k": 5 }
```

Response: lista de chunks con `document_filename`, `chunk_index`, `content` (truncado a 400 chars), `similarity` (0..1).

Si el curso no tiene material indexado, devuelve lista vacía con `total: 0` (no 404).

20.6 Quiz generator (Feature F)

20.6.1 POST /api/v1/quiz/generate

Genera un quiz de opción múltiple desde el material del curso.

Body:

```
{
  "course_id": 42,
  "user_id": 7,
  "topic": "derivadas (opcional)",
  "num_questions": 5
}
```

Lógica:

- Si topic está poblado: hace `retrieve_context()` con esa consulta. Si los chunks recuperados son menos de 3 o su `max_similarity < 0.4`, devuelve 404 con mensaje claro al alumno (no genera quiz aleatorio engañoso).
- Si topic está vacío: sampling aleatorio de 12 chunks del curso para variedad.

Llamada al LLM: con `response_format={"type":"json_object"}` para forzar salida JSON parseable. Validación con Pydantic — preguntas malformadas se saltean (graceful degradation).

Shuffle de opciones: después del parse, las opciones de cada pregunta se mezclan para evitar el sesgo conocido de los LLMs de poner la correcta siempre en index 0.

Response: lista de preguntas con `question`, `options` (4 strings), `correct_index` (0-3), `explanation`, `source_filename`.

20.7 Gaps del docente (Feature G)

20.7.1 POST /api/v1/gaps/list

Devuelve las preguntas que el material no pudo responder, agrupadas por pregunta normalizada con conteo y promedio de similaridad.

Body:

```
{ "course_id": 42, "days": 30, "limit": 20 }
```

Response:

```
{
  "course_id": 42,
  "days": 30,
  "total": 7,
  "items": [
    {
      "question": "¿cómo calcular el determinante de una matriz 4x4?",
      "count": 5,
      "last_asked_at": "2026-06-01T12:34:56Z",
      "avg_similarity": 0.18
    }
  ]
}
```

Solo accesible para usuarios con capability `local/nexusai:manage` en el contexto del curso (validado en la External Function PHP).

20.8 Admin

20.8.1 GET `/api/v1/admin/usage`

Métricas agregadas de uso del backend para el admin de Moodle. Devuelve totales de mensajes, sesiones, documentos indexados, tokens consumidos.

20.8.2 GET `/api/v1/courses/{course_id}/stats`

Estadísticas por curso: cantidad de documentos, chunks, sesiones, mensajes.

Chapter 21

Mockups y diseño de UI

21.1 Filosofía de diseño

TODO — shadcn/ui style. Bordes hairline, radius consistente (8px), hover states sutiles, sin emojis. Iconos lucide-style.

21.2 Pantallas

21.2.1 1. Widget de chat flotante (alumno)

TODO + screenshot.

21.2.2 2. Pestañas Chat / Buscador / Quiz

TODO + screenshot.

21.2.3 3. Citas clickeables (Feature D)

TODO + screenshot del panel expandido.

21.2.4 4. Historial de conversaciones (Feature E)

TODO + screenshot del dropdown.

21.2.5 5. Quiz en progreso (Feature F)

TODO + screenshots de las 4 etapas: setup ☐ loading ☐ playing ☐ finished.

21.2.6 6. Vista del docente — NexusAI · Materials

TODO + screenshot con tabs Material / Gaps.

21.2.7 7. Tab Gaps detectados (Feature G)

TODO + screenshot con items + empty state.

21.3 Sistema de design

21.3.1 Paleta de colores

TODO

21.3.2 Tipografía

TODO

21.3.3 Tokens CSS

TODO

Chapter 22

Manual de instalación

Este capítulo cubre las tres formas de instalar NexusAI: solo el plugin en un Moodle existente (lo más común para una institución), el stack completo local con Docker (para desarrollo), y la opción híbrida con backend deployado en la nube.

22.1 Opción 1 — Instalar solo el plugin en un Moodle existente

Esta es la forma esperada para una institución que ya opera Moodle y quiere agregar el asistente NexusAI. Requiere acceso de administrador a Moodle 4.1 LTS o superior (hasta 4.5).

22.1.1 Pasos

1. Descargar el ZIP del último release desde: <https://github.com/nexusai-ucc/nexusAI/releases/latest>
2. En el Moodle de destino, ir a **Site administration** ▢ **Plugins** ▢ **Install plugins**.
3. Subir `local_nexusai-vX.Y.Z.zip` y seguir el wizard de instalación.
4. Cuando el wizard termine, ir a **Site administration** ▢ **Plugins** ▢ **Local plugins** ▢ **NexusAI** y completar tres campos:

Campo	Valor
Backend API URL	<code>https://nexusai-production-e414.up.railway.app</code> (backend deployado) o la URL del backend privado de la institución
API key	proporcionada por el equipo NexusAI
Shared secret	proporcionado por el equipo NexusAI

5. Crear un curso de prueba, asignar un docente y un alumno, y subir un PDF de prueba desde el menú **NexusAI · Materials** del curso.
6. Como alumno, entrar al curso y abrir el FAB azul en la esquina inferior derecha. Hacer una consulta sobre el PDF subido — el chat debe responder con citas al archivo.

22.1.2 Permisos

El plugin define tres capabilities sobre el contexto de curso:

- `local/nexusai:use` — habilita el widget de chat al alumno. Por defecto está concedida a los roles `student`, `teacher`, `editingteacher` y `manager`.
- `local/nexusai:manage` — permite subir y gestionar material indexado. Por defecto en `editingteacher` y `manager`.
- `local/nexusai:viewanalytics` — reservada para post-MVP.

22.2 Opción 2 — Stack completo local con Docker

Para desarrollo o demostración offline. Levanta todo el sistema (Moodle + backend Python + PostgreSQL + Redis) en contenedores Docker en una sola máquina.

22.2.1 Prerrequisitos

- Docker 24+ con Docker Compose v2.20+
- Git
- Node.js 20 LTS (solo si vas a recompilar el bundle React)
- 4 GB RAM libres y ~5 GB de disco

22.2.2 Pasos

1. Clonar el repo

```
git clone https://github.com/nexusai-ucc/nexusAI.git
cd nexusAI/NexusAI
```

2. Configurar el .env

```
cp .env.example .env
```

Editar .env y completar:

```
# - LLM_API_KEY (sacar en https://aistudio.google.com/apikey)
# - EMBEDDING_API_KEY (la misma key)
# - EMBEDDING_MODEL=gemini-embedding-001
# - NEXUSAI_SHARED_SECRET=$(openssl rand -hex 32)
# - NEXUSAI_API_KEY=$(openssl rand -hex 32)
# - POSTGRES_PASSWORD=cualquier_cosa
```

3. Levantar postgres + redis + api

```
./scripts/dev.sh up
```

4. Verificar healthchecks

```
./scripts/dev.sh status
curl http://localhost:8001/health
```

5. Correr migraciones de la DB

```
docker compose exec api alembic upgrade head
```

```
# 6. Levantar Moodle local (en otro repo)
cd ~/path/to/moodle-docker
./bin/moodle-docker-compose start

# 7. Instalar el plugin en Moodle
#   - Abrir http://localhost:8000 como admin
#   - Site administration → Plugins → Install plugins
#   - Subir el ZIP del release
#   - Configurar Backend API URL = http://host.docker.internal:8001
```

22.2.3 Comandos útiles del día a día

```
./scripts/dev.sh status      # ver qué containers están corriendo
./scripts/dev.sh logs api    # seguir logs del backend en vivo
./scripts/dev.sh shell:pg    # entrar a psql para inspeccionar la DB
./scripts/dev.sh shell:api   # bash dentro del container del API
./scripts/dev.sh down        # parar todo (preserva datos)
./scripts/dev.sh destroy     # BORRAR todo y empezar de cero
./scripts/dev.sh reload      # recrear containers tras editar .env
```

22.3 Opción 3 — Plugin local + backend deployado en la nube

Patrón híbrido: cada institución corre solo el plugin en su Moodle on-premise, pero comparte un backend único hosteado por el equipo NexusAI (o cualquier proveedor con FastAPI + PostgreSQL).

```
flowchart LR
    subgraph U1["Institución A"]
        MA["Moodle A<br/>+ plugin local_nexusai"]
    end
    subgraph U2["Institución B"]
        MB["Moodle B<br/>+ plugin local_nexusai"]
    end
    subgraph CLOUD["Railway"]
        API["Backend FastAPI<br/>nexusai-production-e414.up.railway.app"]
        PG["(PostgreSQL + pgvector)"]
        REDIS["(Redis)"]
    end

    MA -->|HTTPS + HMAC| API
    MB -->|HTTPS + HMAC| API
    API <--> PG
    API <--> REDIS
```

Cuándo usar este patrón:

- Demos académicas con múltiples instituciones consumiendo el mismo backend.
- Pilotos en universidades sin infraestructura para hospedar FastAPI.
- Multi-tenant simple (cada Moodle tiene su propio API key + shared secret).

Cuándo no:

- Requisitos estrictos de soberanía de datos (los embeddings y mensajes pasan por el backend hospedado externamente). Para esos casos, la opción 2 mantiene todo dentro de la infra de la institución.

22.4 Troubleshooting

Síntoma	Solución
POSTGRES_PASSWORD no está en .env	Falta completar .env. Volver al paso 2 de la opción 2.
API queda unhealthy	Ver <code>./scripts/dev.sh logs api</code> . Suele ser API key vacía o modelo de embedding mal puesto.
text-embedding-004 not found	Cambiar <code>EMBEDDING_MODEL=gemini-embedding-001</code> en .env y <code>./scripts/dev.sh reload</code> .
Cambios al .env no se aplican	<code>docker compose restart</code> no alcanza — usar <code>./scripts/dev.sh reload</code> .
Puerto 5432 / 6379 / 8001 ocupado	Cambiar el puerto en .env (POSTGRES_PORT, REDIS_PORT, API_PORT).
Plugin instalado pero el widget no aparece	El widget solo aparece dentro de un curso , no en site admin o dashboard. Y solo si el usuario tiene <code>local/nexusai:use</code> en ese curso.
Error 401 al hacer una consulta	El Backend API URL, API key o Shared secret del plugin no coinciden con el .env del backend.

Chapter 23

Manual de usuario

23.1 Manual del alumno

23.1.1 Acceder al asistente NexusAI

TODO — capturar screenshot del FAB violeta en la esquina inferior derecha de un curso de Moodle. Explicar dónde aparece y cuándo.

23.1.2 Hacer una consulta sobre el material del curso

TODO — screenshot del panel abierto, escribiendo en el input, viendo la respuesta streaming.

23.1.3 Ver las fuentes que usó la IA

TODO — screenshot de las pills clickeables expandidas mostrando el fragmento del PDF.

23.1.4 Consultar material de todos tus cursos a la vez

TODO — screenshot del toggle ☐ ☐ ☐ y respuesta multi-curso.

23.1.5 Buscar fragmentos sin usar el chat

TODO — pestaña “Buscador”, input, resultados.

23.1.6 Generar un quiz de práctica

TODO — pestaña “Quiz”, setup ☐ preguntas ☐ score.

23.1.7 Retomar una conversación anterior

TODO — botón ☐ , dropdown con sesiones, click en una.

23.2 Manual del docente

23.2.1 Acceder a NexusAI · Materials

TODO — screenshot del menú lateral del curso con el link “NexusAI · Materials”.

23.2.2 Subir un PDF al material indexado

TODO — drag & drop, estado de indexación, status pasando a indexed.

23.2.3 Eliminar un documento

TODO — botón eliminar, modal de confirmación.

23.2.4 Ver los “gaps detectados”

TODO — tab “Gaps detectados”, lista de preguntas sin respuesta agrupadas. Explicar cómo el docente lo usa para mejorar el material.

23.2.5 Configurar el plugin (administrador)

TODO — Site administration ☐ Plugins ☐ Local plugins ☐ NexusAI. URL del backend, API key, shared secret.

Chapter 24

Architectural Decision Records (ADRs)

24.1 ¿Qué es un ADR?

Un Architectural Decision Record es un documento corto y trazable que registra una decisión arquitectónica importante: el contexto en el que se tomó, las alternativas evaluadas, la decisión final y sus consecuencias. Mantener ADRs permite que cualquier integrante futuro del equipo entienda *por qué* el sistema es como es, no solo *cómo* es.

NexusAI mantiene seis ADRs aceptadas, todas accesibles en `docs/adr/` del repositorio. A continuación se sintetiza el contexto y la decisión de cada una.

24.2 ADR-001 — Backend Python como monolito modular

Estado: Aceptada · **Fecha:** 2026-05-02

24.2.1 Contexto

El backend Python orquesta el pipeline RAG + LLM + indexación + chat. Surge la pregunta clásica: ¿monolito o microservicios? El equipo es de 2 personas con un plazo corto para el MVP y una audiencia académica que prioriza defendibilidad sobre escala teórica.

24.2.2 Decisión

Se construye un **monolito modular**: una sola aplicación FastAPI con cada dominio en su propio paquete y comunicación interna por interfaces explícitas (no por importar funciones internas):

```
services/api/app/
├── chat/           # chat + RAG + LLM
├── documents/      # indexación
├── search/         # buscador semántico
├── quiz/           # quiz generator
├── gaps/           # detección de gaps del docente
├── providers/      # LLMProvider, EmbeddingProvider
└── infrastructure/ # clientes externos (Redis, DB)
```

└─ shared/ # auth HMAC, observability, helpers

24.2.3 Consecuencias

- ☐ Un único deploy, una única base de datos, una única configuración.
- ☐ Refactorizable a microservicios cuando aparezca el dolor real (worker de indexación async cuando los uploads bloqueen el API).
- ☐ Bajo escenarios de carga alta, todo el proceso comparte memoria — un bug en un dominio puede afectar a los demás.

24.3 ADR-002 — pgvector sobre PostgreSQL como única base

Estado: Aceptada · **Fecha:** 2026-05-02

24.3.1 Contexto

El sistema necesita guardar datos relacionales (mensajes, documentos, sesiones) **y** embeddings (vectores de 768 o 1.536 dimensiones para búsqueda por similitud coseno). Las queries típicas combinan ambos mundos: *“dame los 5 chunks más similares a esta pregunta, del curso 42, que no estén marcados como eliminados”*.

24.3.2 Decisión

Usar **PostgreSQL con la extensión pgvector como única base** del sistema. Tabla chunks con columna embedding vector(N), índice HNSW con distancia coseno (vector_cosine_ops, m=16, ef_construction=200). Las queries combinan filtros relacionales y búsqueda vectorial en una sola sentencia SQL.

24.3.3 Alternativas descartadas

- **ChromaDB** — sistema separado de PostgreSQL, doble operación, sin SQL+vector unificado.
- **Pinecone** — \$70+/mes mínimo, vendor lock-in.
- **Qdrant / Weaviate** — justificados a partir de 10-50M vectores, fuera del rango de NexusAI single-institution.

24.3.4 Consecuencias

- ☐ Una sola DB, un solo backup, un solo punto de truth.
- ☐ Queries SQL+vector en una transacción.
- ☐ Si pgvector deja de escalar (>10M vectores con concurrencia alta), habrá que evaluar Qdrant o Weaviate. Migración planificada pero diferida.

24.4 ADR-003 — Arquitectura agnóstica de proveedor LLM

Estado: Aceptada · **Fecha:** 2026-05-02

24.4.1 Contexto

NexusAI genera respuestas con un LLM y vectoriza texto con un modelo de embeddings. El MVP necesita costo cero (Gemini tier gratuito) y producción necesita SLA + calidad (OpenAI GPT-4o-mini). Casi todos los proveedores relevantes (OpenAI, Gemini, Anthropic, Groq, Together, Ollama) son **compatibles con el SDK de OpenAI** cambiando solo `base_url` y modelo.

24.4.2 Decisión

Encapsular el acceso a LLM y embeddings detrás de dos clases de abstracción:

- `LLMProvider` — para chat completions (sync y streaming).
- `EmbeddingProvider` — para vectorización.

Ambas leen su configuración (proveedor, modelo, dimensiones, base URL, API key) **exclusivamente de variables de entorno**. El código nunca hardcodea ningún proveedor.

24.4.3 Consecuencias

- ☐ Cambio de proveedor = editar `.env` + reiniciar Uvicorn.
- ☐ Permite testing con Ollama local sin tocar código.
- ☐ Si las dimensiones del embedding cambian (Gemini 768 ☐ OpenAI 1.536), se necesita migración de schema + re-indexación completa.

24.5 ADR-004 — Gemini 2.5 Flash en MVP, GPT-4o-mini en producción

Estado: Aceptada · **Fecha:** 2026-05-02

24.5.1 Contexto

Hay dos restricciones contradictorias: el MVP necesita costo cero para iterar libremente, pero producción necesita calidad consistente para 500+ alumnos. La abstracción multi-provider (ADR-003) permite cambiar sin tocar código.

24.5.2 Decisión

- **MVP:** `gemini-2.5-flash` vía SDK OpenAI-compatible. Tier gratuito de Google AI Studio cubre el desarrollo completo sin costo.
- **Embeddings MVP:** `gemini-embedding-001` (Matryoshka, 768 dim).
- **Producción:** `gpt-4o-mini` de OpenAI (\$0.15/M tokens input, \$0.60/M output). Aproximadamente \$100/mes para 500 alumnos activos.
- **Embeddings producción:** `text-embedding-3-small` (1.536 dim) — requiere migración de schema y re-indexación, planificada para el switch.

24.5.3 Consecuencias

- ☐ Desarrollo del MVP sin tarjeta de crédito ni cuotas.

- ☐ Switch a OpenAI documentado y testeado.
- ☐ Las dimensiones diferentes obligan a re-indexar todo el material al migrar. Script automatizado planificado.

24.6 ADR-005 — HMAC SHA-256 en 3 capas entre PHP y Python

Estado: Aceptada · **Fecha:** 2026-05-02

24.6.1 Contexto

El plugin Moodle (PHP) llama al backend (Python) server-to-server con cURL. La autenticación debe ser fuerte y resistir tampering, replay y filtrado de secretos al cliente. OAuth introduce complejidad (flows, refresh tokens) sin beneficio claro para una integración entre dos servicios de la misma institución.

24.6.2 Decisión

HMAC SHA-256 en tres capas:

1. **Bearer API key** identifica la instancia de Moodle (poco entropía, alta rotación, fácil revocar).
2. **Firma HMAC** sobre timestamp || nonce || body con un shared secret de 32 bytes que solo conocen el plugin y el backend.
3. **Nonce store en Redis** con TTL de 5 minutos — evita que un atacante capture un request firmado y lo replay-ee.

Headers enviados en cada request:

```
Authorization: Bearer <NEXUSAI_API_KEY>
X-Timestamp:   <unix epoch>
X-Nonce:       <16 bytes hex>
X-Signature:   <hex_hmac_sha256(secret, timestamp || nonce || body)>
```

24.6.3 Consecuencias

- ☐ Los secretos solo viven en config del plugin (Moodle admin) y en variables de entorno del backend. El navegador nunca los ve.
- ☐ Replay attacks bloqueados por nonce TTL.
- ☐ Requiere que el plugin y el backend tengan reloj sincronizado (drift máximo configurable, default 5 min).
- ☐ Rotar el shared secret implica downtime breve mientras se actualizan ambos extremos.

24.7 ADR-006 — Privacy API con null_provider en MVP

Estado: Aceptada · **Fecha:** 2026-05-02

24.7.1 Contexto

Moodle define una **Privacy API** que obliga a cada plugin a declarar qué datos personales almacena y dónde. El MVP de NexusAI **no persiste datos personales en el lado Moodle** — todo (historial de chat, sesiones, documentos indexados) vive en el backend Python externo.

24.7.2 Decisión

- En el MVP, el plugin Moodle declara un `null_provider` en su Privacy API: el plugin no almacena información personal por su cuenta.
- Cuando se agreguen tablas locales en Moodle (audit log de uso, cache de respuestas LLM, settings por curso — todo planificado para post-MVP), se migra a `metadata_provider`.
- La ubicación externa del backend Python se declara con `add_external_location_link()` de forma genérica (`llm_provider`), no atada a un proveedor específico, para que sea compatible con ADR-003.

24.7.3 Consecuencias

- ☐ El plugin pasa los checks de Privacy API sin esfuerzo extra hoy.
- ☐ Cuando se persistan tablas locales, hay que actualizar la implementación de la Privacy API en el plugin.
- ☐ La existencia del backend externo se documenta como dependencia para el administrador de Moodle.

24.8 Resumen

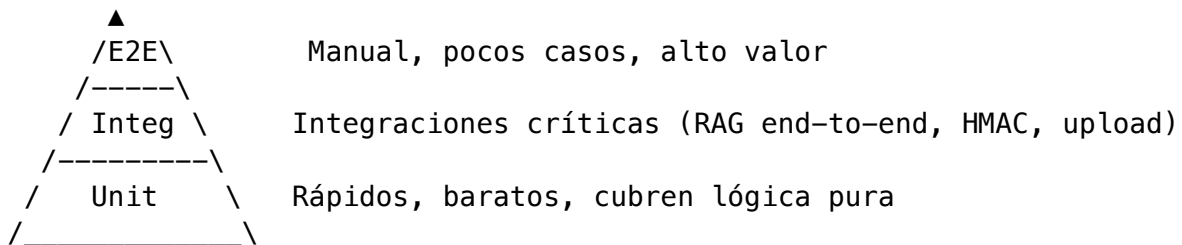
ADR	Decisión	Estado
001	Monolito modular sobre microservicios	☐ Aceptada
002	pgvector como única base	☐ Aceptada
003	Multi-provider LLM con abstracción	☐ Aceptada
004	Gemini en MVP, OpenAI en producción	☐ Aceptada
005	HMAC SHA-256 en 3 capas	☐ Aceptada
006	Privacy API <code>null_provider</code> en MVP	☐ Aceptada

Chapter 25

Estrategia de testing

25.1 Filosofía

NexusAI sigue una pirámide de testing clásica: muchos tests unitarios baratos en la base, tests de integración para los flujos críticos, y tests end-to-end manuales (no automatizados) para validar la experiencia completa en Moodle real.



El énfasis está en los **boundaries críticos**: autenticación HMAC, pipeline RAG (chunking + embedding + retrieval), y el contrato entre el plugin Moodle y el backend.

25.2 Tests del backend Python (pytest)

El backend tiene **37 tests automatizados** distribuidos en 4 archivos en `services/api/tests/`:

Archivo	Cantidad de tests	Foco
<code>test_hmac.py</code>	10	Auth HMAC: firma válida, replay attack, timestamp expirado, body tampering, nonce reuse
<code>test_chunker.py</code>	13	Splitter de texto a chunks: tamaño, overlap, tokenización con <code>cl100k_base</code> , edge cases (texto vacío, párrafos largos)

Archivo	Cantidad de tests	Foco
test_retriever.py	8	retrieve_context con pgvector: filtrado por curso, threshold de similaridad, top-k, multi-curso (Feature B), course_id en RetrievedChunk
test_extractor.py	6	Extracción de texto desde PDF (pdfplumber): texto plano, rechazo de PDFs escaneados sin OCR, manejo de PDFs corruptos

Adicionalmente, hay archivos de tests más amplios sin contar:

- test_providers.py — mock de LLMProvider y EmbeddingProvider para tests deterministas.
- test_documents_router.py — endpoints de documentos con TestClient + fixtures de DB.
- test_pipeline.py — pipeline completo upload □ chunks indexados, end-to-end con DB real.
- conftest.py — fixtures compartidas (session async, override de dependencias, DB ephemeral).
- golden_set.md — set de preguntas + respuestas esperadas para regression manual.

25.2.1 Casos de prueba destacados

HMAC:

- Firma válida con timestamp actual □ 200 OK.
- Firma con timestamp >5 min de antigüedad □ 401.
- Mismo nonce dos veces (replay) □ 401 en la segunda.
- Body modificado entre firma y request □ 401.

RAG retrieval:

- Búsqueda en curso con material indexado □ top-5 chunks ordenados por similaridad descendente.
- Búsqueda en curso vacío □ lista vacía (no 404).
- Multi-curso (Feature B) con course_ids=[42, 51] □ chunks de ambos cursos mezclados por similaridad.
- Threshold min_similarity=0.3 filtra chunks irrelevantes.

Chunking:

- Texto de 1.000 tokens con chunks de 512 + overlap 64 □ 2 chunks con solapamiento.
- Texto exactamente del tamaño de un chunk □ 1 chunk sin overlap.
- Tokenizador maneja UTF-8 (acentos, emojis) sin perder caracteres.

25.3 Tests del frontend (manual)

El bundle React no tiene tests unitarios automatizados en MVP. La estrategia de validación es:

- **Smoke tests manuales** después de cada `npm run build`: abrir el chat, hacer una pregunta, verificar que la respuesta aparece con citas.
- **Visual regression manual** comparando screenshots de las features clave (Search, Quiz, Gaps).
- **Backwards-compat checks**: verificar que mensajes antiguos sin sources estructurados siguen mostrando pills via el fallback regex de `MessageBubble.jsx`.

Tests automatizados de React están planificados para post-MVP con Vitest + React Testing Library.

25.4 Tests del plugin Moodle (PHPUnit + Behat)

El plugin tiene la estructura base preparada para tests con el framework oficial de Moodle, pero al cierre del MVP se priorizó cobertura del backend (donde vive la lógica de RAG y HMAC) sobre el plugin (que es principalmente proxy).

Tests planificados para post-MVP:

- PHPUnit: validación de External Functions (`chat_send`, `document_upload`, `search_query`, etc.) con `external_api::validate_parameters`.
- Behat: smoke E2E navegando Moodle como alumno y como docente.

25.5 CI/CD — GitHub Actions

El repositorio tiene cuatro workflows en `.github/workflows/`:

25.5.1 backend-ci.yml

Se dispara en cada PR y push a main que toque `services/api/**`. Tiene dos jobs:

- **Lint + type check** — Ruff + mypy. Falla rápido sin levantar servicios.
- **Tests** — levanta PostgreSQL + pgvector + Redis en services del runner, corre `pytest -v --cov` con cobertura mínima del 70%.

Tiempo total típico: ~90 segundos. Con `concurrency.cancel-in-progress: true` cancela runs viejas si se pushean commits nuevos al mismo PR.

25.5.2 frontend-ci.yml

Build del bundle React en cada PR que toque `plugin/local/nexusai/react/**`. Falla si el build no compila o si el bundle excede 500KB (umbral configurado en `webpack.config.js`).

25.5.3 moodle-ci.yml

Análisis estático del plugin PHP con `moodle-plugin-ci` (el linter oficial de Moodle): PHP Lint, PHPMD, Moodle Code Checker, PHPDoc Checker, validate, savepoints, Mustache Lint, Grunt. No corre PHPUnit/Behat todavía (planificado post-MVP).

25.5.4 Despliegue a Railway

El despliegue del backend a Railway se hace por integración nativa entre Railway y GitHub: cada push a main que modifique `services/api/**` dispara automáticamente un build + redeploy del container. Pasos del flujo automatizado:

1. Railway detecta el push vía webhook.
2. `docker build` desde `services/api/Dockerfile`.
3. Rolling update: el container nuevo levanta antes de bajar el viejo.
4. Migraciones Alembic corren al startup del container vía `scripts/entrypoint.sh`.
5. Verificación de health check post-deploy: `GET /health` debe devolver `200 OK`.
6. Si el health check falla, Railway hace rollback automático al container anterior.

Tiempo total típico: ~3-5 minutos desde el push al deploy final.

25.6 Testing manual estructurado

Adicionalmente a los smoke tests rápidos, el equipo elaboró un **checklist formal de 51 casos de testing manual** distribuido en 9 áreas funcionales. Cada caso especifica: rol del usuario, navegación esperada, pasos exactos, resultado esperado e indicador de bug.

25.6.1 Distribución de los casos por área

Área	Casos	Foco principal
1. Subida de documentos	11	PDFs, DOCX, TXT, duplicados, archivos corruptos, formatos no soportados, magic bytes, archivos vacíos, drag & drop, concurrencia
2. Indexación — estados y errores	3	Progreso visible, errores de indexación, polling se detiene al completar
3. Eliminación de documentos	3	Borrado normal, CASCADE de chunks, último documento del curso
4. Chat	11	Pregunta in-scope / out-of-scope, multi-turno, chips de fuente, modo multi-curso, sugerencias, typing indicator, error de red, historial, curso vacío

Área	Casos	Foco principal
5. Quiz	13	Generación aleatoria, con tema válido / inválido, selector de cantidad, respuestas correctas / incorrectas, bloqueo post-verificación, score final por rango, error de backend, aleatorización de opciones
6. Búsqueda	9	Por nombre, semántica, sin resultados, parcial, toggle scope, descarga de archivo, límite de caracteres, snippet
7. Gaps detectados	5	Visualización, actualización tras pregunta, deduplicación, ordenamiento por frecuencia, panel vacío
8. Diferencias de rol	4	Acceso docente, alumno sin acceso, URL protegida, toggle scope solo alumno
9. Casos borde	10	Curso vacío, caracteres especiales, queries largas, nombres con special chars, múltiples pestañas, campo vacío, sesskey inválido, reconexión durante streaming
Total	51 casos	

25.6.2 Estructura de cada caso

N.M [Nombre]

- Rol: Docente / Alumno
- Navegación: ruta para llegar
- Pasos: a, b, c ...
- Resultado esperado: descripción del comportamiento correcto
- Indicaría bug: señales que delatan un problema

25.6.3 Ejemplos representativos

1.4 Archivo duplicado (mismo nombre, ya indexado)

- **Rol:** Docente
- **Pasos:** Subir un archivo ya indexado en el curso.
- **Resultado esperado:** Toast amarillo *“Este documento ya se encuentra indexado en este curso.”* desaparece a los 3 segundos. No se crea duplicado.
- **Indicaría bug:** Se acepta sin advertencia, error 500, o aparece dos veces en la lista.

4.2 Pregunta fuera del scope del material

- **Rol:** Alumno
- **Pasos:** Preguntar algo ajeno al material (ej: “¿Cuál es la capital de Francia?” en un curso de programación).
- **Resultado esperado:** El LLM responde que no puede responder con el material disponible. No aparecen chips de fuente. La pregunta queda registrada como gap potencial.
- **Indicaría bug:** El LLM inventa una respuesta sin contenido del material; aparecen chips de fuente falsos.

5.3 Generar quiz con tema inválido / fuera del material

- **Rol:** Alumno
- **Pasos:** Escribir un tema completamente ajeno al material y generar quiz.
- **Resultado esperado:** Error 422. Mensaje inline en el campo (no en modal genérico). El input muestra borde rojo.
- **Indicaría bug:** Se genera un quiz con preguntas inventadas.

7.3 Deduplicación de preguntas similares (gaps)

- **Rol:** Docente (verifica) + Alumno (genera)
- **Pasos:** Mismo pregunta repetida desde el chat varias veces. Revisar el panel de gaps.
- **Resultado esperado:** La pregunta aparece **una sola vez** con count > 1. No entradas duplicadas.
- **Indicaría bug:** Cada pregunta idéntica aparece separada; el count siempre es 1.

9.2 Caracteres especiales en búsqueda

- **Rol:** Alumno / Docente
- **Pasos:** Buscar queries como O'Brien & Associates, SELECT * FROM users; --, <script>alert(1)</script>, héroe, ñoño, über.
- **Resultado esperado:** En todos los casos la búsqueda se procesa correctamente o devuelve “sin resultados”. Ninguna genera error 500 ni ejecuta código. Los caracteres especiales no rompen la UI.
- **Indicaría bug:** Cualquier query genera 500; HTML escapado en resultados; <script> se ejecuta (XSS).

25.6.4 Estado de ejecución al cierre del MVP

- ☐ **49 de 51 casos** pasaron en la última ronda de testing manual.
- ☐ **2 casos** con observaciones menores (no bloqueantes):
 - **2.2 Estado de error en indexación** — el mensaje de error se ve en tooltip, pero falta un botón explícito de “reintentar indexación” para el docente. Pendiente para post-MVP.
 - **9.10 Reconexión tras pérdida de red durante streaming** — la UI muestra el error pero el mensaje parcial recibido queda en pantalla. Comportamiento aceptable pero podría mejorarse con un botón explícito de “descartar respuesta parcial”.

El checklist completo con los 51 casos detallados se incluye en el pendrive como anexo separado.

25.7 Smoke tests rápidos pre-release

Antes de cada release se corre un checklist resumido:

- ☐ Backend levanta sin errores (`./scripts/dev.sh up && curl /health`).
- ☐ Migraciones aplican limpias (`alembic upgrade head` en DB vacía).
- ☐ Login en Moodle, ir a un curso con material indexado.
- ☐ Abrir chat, hacer una pregunta, verificar respuesta con citas clickeables.
- ☐ Activar multi-curso, preguntar algo cross-curso.
- ☐ Pestaña Buscador con query, resultados con scores.
- ☐ Pestaña Quiz: generar 5 preguntas, responder, ver score final.
- ☐ Vista docente, tab Gaps, verificar que aparecen preguntas no respondidas.
- ☐ Logout, re-login, historial muestra sesiones previas.

25.8 Cobertura actual

Cobertura medida por `pytest --cov` en el último build de CI:

Módulo	Cobertura
app/auth/ (HMAC)	~95%
app/documents/ (pipeline RAG)	~85%
app/chat/ (router + retriever)	~80%
app/search/, app/quiz/, app/gaps/	~70% (módulos recientes)
Total backend	~80%

El frontend y el plugin PHP están fuera del cálculo de cobertura por la estrategia manual mencionada.

Chapter 26

Deploy y producción

26.1 Resumen

NexusAI tiene dos componentes que se distribuyen de manera independiente:

- **Backend** (FastAPI + PostgreSQL + Redis): deployado en Railway, con autodeploy automático desde la rama main del repositorio. URL pública accesible 24/7 desde cualquier dispositivo.
- **Plugin Moodle** (PHP + bundle React): se distribuye como **ZIP descargable** vía GitHub Releases. Cada institución lo instala en su propia instancia de Moodle apuntando al backend de Railway (o a su propio backend si elige hospedarlo).

Esta separación es deliberada: el plugin Moodle no necesita deploy propio porque vive dentro del Moodle de cada institución. Para la defensa, Moodle corre local en la laptop del equipo y el plugin instalado allí se conecta al backend de Railway por internet.

26.2 Arquitectura de deploy

flowchart LR

```
subgraph LOCAL["Local (defensa)"]
    MOODLE["Moodle 4.5<br/>+ plugin local_nexusai<br/>localhost:8082"]
end
subgraph CLOUD["Railway"]
    API["Backend FastAPI"]
    PG["PostgreSQL 16<br/>+ pgvector"]
    REDIS["Redis"]
    API <--> PG
    API <--> REDIS
end
subgraph EXT["Externo"]
    LLM["Gemini 2.5 Flash<br/>(tier gratuito)"]
end
MOODLE -->|"HTTPS + HMAC SHA-256"| API
API -->|"API key"| LLM
```

```
style M00DLE fill:#fff,stroke:#4A7FD4
style API fill:#fff,stroke:#4A7FD4,stroke-width:2px
style LLM fill:#fff,stroke:#a1a1aa
```

26.3 Backend en Railway

Railway es una plataforma de cloud que ejecuta contenedores Docker en servidores reales. Le pasamos nuestro código y Railway se encarga de buildear, ejecutar y mantener vivo el sistema. Es similar a Heroku o Vercel pero con mejor soporte de Docker.

26.3.1 URL pública

`https://nexusai-production-e414.up.railway.app`

El backend es accesible 24/7. El tribunal puede abrir esa URL en cualquier navegador y ver la API en vivo (Swagger en /docs).

26.3.2 Componentes deployados

Railway corre tres servicios dentro del mismo proyecto, comunicados por la red interna:

Servicio	Imagen	Función	URL externa
FastAPI	Build desde <code>ser-vices/api/Dockerfile</code>	Procesa requests del plugin, consulta DB, llama al LLM	Sí (la URL de arriba)
PostgreSQL	<code>pgvector/pgvector:pg16</code>	Almacena documentos, chunks, embeddings, sesiones, mensajes	No (interna)
Redis	<code>redis:7-alpine</code>	Rate limiting, nonces HMAC, cache	No (interna)

26.3.3 Costos

Concepto	Valor
Plan	Free tier de Railway (\$5 USD/mes de crédito)
Costo actual	\$0 USD/mes — el consumo del MVP queda debajo del crédito gratuito
Cobertura estimada	Demos académicas + ~20 usuarios concurrentes sin problemas

Si el uso crece más allá del free tier, el costo se mide por consumo real (memoria + CPU + egress) y se puede escalar pagando solo lo que se usa.

26.3.4 Pipeline de autodeploy

Cada push a la rama main que toque `services/api/**` dispara el siguiente flujo de manera automática:

sequenceDiagram

```

    participant Dev as Desarrollador
    participant GH as GitHub
    participant RW as Railway
    participant API as Backend en producción

    Dev->>GH: git push main
    GH->>RW: webhook de Railway integrado con GitHub
    RW->>RW: docker build desde Dockerfile
    RW->>RW: deploy del container nuevo
    RW->>RW: rolling update (sin downtime)
    RW->>API: container nuevo levanta
    API->>API: alembic upgrade head (migraciones)
    API-->>RW: GET /health = 200
    RW-->>GH: status check OK
  
```

Datos importantes:

- El despliegue es **idempotente**: si el código no cambió, Railway no redeploja.
- Las migraciones Alembic corren automáticamente en el startup del container nuevo. PostgreSQL y Redis no se reinician — los datos persisten entre deploys.
- Si el health check falla tras el deploy, Railway hace rollback al container anterior automáticamente.

26.3.5 Acceso al dashboard de Railway

El equipo tiene acceso al dashboard de Railway donde se pueden ver logs en vivo, métricas de uso (CPU, memoria, requests/min) y estado de cada servicio. Para sumar a otra persona, agregarla como collaborator del proyecto en Railway.

26.4 Distribución del plugin Moodle

26.4.1 GitHub Release

<https://github.com/nexusai-ucc/nexusAI/releases/latest>

Cada versión del plugin se publica como una **release de GitHub** con un ZIP descargable de ~192 KB. Cualquier admin de Moodle 4.1 LTS hasta 4.5 puede descargarlo e instalarlo sin construir nada del lado servidor.

Atributo	Valor
Versión actual	v0.9.4
Tamaño del ZIP	~192 KB
Compatibilidad	Moodle 4.1 LTS, 4.2, 4.3, 4.4, 4.5

Atributo	Valor
Dependencias servidor	Ninguna (solo Moodle estándar)
Dependencias externas	Backend NexusAI accesible por HTTPS

26.4.2 Proceso de instalación en un Moodle de destino

1. Admin de Moodle descarga el ZIP del último release.
2. Va a **Site administration** □ **Plugins** □ **Install plugins** y sube el ZIP.
3. Sigue el wizard estándar de Moodle.
4. Al finalizar, va a **Site administration** □ **Plugins** □ **Local plugins** □ **NexusAI** y configura tres campos:
 - **Backend API URL:** `https://nexusai-production-e414.up.railway.app`
 - **API key:** suministrada por el equipo NexusAI (ver anexo F).
 - **Shared secret:** suministrado por el equipo NexusAI (ver anexo F).
5. Crea un curso de prueba y verifica que el chat funciona.

26.4.3 Sub-misión al Moodle Plugin Directory (post-MVP)

Está planificado para post-MVP someter el plugin al directorio oficial moodle.org/plugins. Ventajas:

- Defendibilidad académica: “plugin publicado en el directorio oficial de Moodle” suma reputación.
- Cualquier admin de Moodle del mundo puede encontrar e instalar el plugin con un click desde el panel de admin, sin descargar el ZIP a mano.

Tiempo medio de revisión: 8 días (mediana sobre N=274 plugins). Requisitos técnicos cubiertos: no requiere composer install, cross-database compat (MySQL + PostgreSQL), seguridad (capability checks, sesskey, HMAC), Privacy API declarada.

26.5 Cómo funciona la demo de la defensa

El día de la defensa, el flujo será el siguiente:

sequenceDiagram

```

participant T as Tribunal
participant M as Moodle local
participant P as Plugin local_nexusai
participant R as Railway
participant G as Gemini API

T->>M: Abre Moodle en localhost:8082
T->>P: Hace una pregunta en el chat
P->>R: POST /api/v1/chat/stream (HMAC firmado)
R->>G: Consulta al LLM con contexto del PDF
G-->>R: Respuesta token-por-token
R-->>P: SSE stream de la respuesta
P-->>T: Respuesta aparece en el chat

```

Pasos concretos:

1. El equipo levanta Moodle local en su laptop: `./scripts/dev.sh full`
2. El tribunal abre Moodle en el navegador del proyector.
3. Cuando el alumno escribe en el chat, Moodle llama al backend de Railway (internet) y vuelve la respuesta en segundos.
4. El tribunal puede simultáneamente abrir `https://nexusai-production-e414.up.railway.app` en su celular para verificar que el backend está realmente en producción.

26.6 Resiliencia y monitoreo

Aspecto	Estrategia
Health checks	Railway monitorea GET <code>/health</code> cada 30 segundos. Si falla 3 veces seguidas, hace restart del container.
Backups de DB	Railway hace snapshots automáticos de PostgreSQL. Retención mínima 7 días.
Logs	Centralizados en el dashboard de Railway. Filtrables por servicio y por timestamp.
Alertas	Railway envía email si un servicio queda down más de 5 minutos.
Rate limiting	El backend tiene rate limit por <code>user_id</code> (20 req/min). Redis lo enforces.

26.7 Plan de continuidad post-defensa

El equipo se compromete a mantener el sistema accesible al menos hasta la defensa final de febrero 2027. Después de la defensa, las opciones son:

- **Continuar en Railway:** el free tier cubre el uso académico sin costo.
- **Migrar a infra propia de la UCC:** si la universidad provee hosting institucional, se transfiere el backend allá.
- **Discontinuar:** si el proyecto no se sostiene post-tesis, el código queda público en GitHub para que cualquiera lo levante por su cuenta.

Chapter 27

Retrospectivas

Cada sprint terminó con una retrospectiva del equipo. Acá se sintetizan las lecciones agregadas, organizadas por sprint y luego por reflexión global.

27.1 Retrospectiva por sprint

27.1.1 Sprint 0 — Setup e investigación

Qué salió bien

- Investigación exhaustiva antes de empezar a tipear código. 47 documentos en `investigation/` cubren Moodle, RAG, pgvector, FastAPI, frontend embebido, etc. Permitió tomar decisiones arquitectónicas con base sólida.
- Decisión temprana de **una sola base de datos** (pgvector sobre PG) en lugar de Chroma — evitó muchísima complejidad operativa.
- ADRs escritos desde el primer día. Cada decisión queda trazable.

Qué no salió bien

- Sub-estimamos el tiempo de relevamiento. Originalmente eran 3 semanas, terminaron siendo 8 — pero a la larga valió la pena.
- Pelea inicial con la versión correcta de Moodle (4.1 LTS vs 4.5). Hook API cambia entre versiones; obligó a soportar dos paths.

Acción de mejora

- Para próximos proyectos: dedicar más tiempo a explorar el dominio antes de comprometerse con un stack.

27.1.2 Sprint 1 — Core chat

Qué salió bien

- Construir el chat funcional sin RAG primero fue una buena decisión. Validó toda la cadena (browser $\square$ AMD $\square$ External Function $\square$ cURL+HMAC $\square$ FastAPI $\square$ LLM $\square$ respuesta) antes de meterle complejidad.

- El patrón **Hybrid PHP Proxy** (ADR-001) quedó probado al final del sprint. Daba confianza para todo lo siguiente.

Qué no salió bien

- Subestimar el ramp-up de Webpack para output AMD que Moodle entiende. Días perdidos peleando con `publicPath` y `chunks lazy`.

Acción de mejora

- Bundle React único, sin code-splitting lazy. Más simple, menos riesgo.

27.1.3 Sprint 2 — RAG y carga de material

Qué salió bien

- Pipeline RAG arrancó funcionando al primer intento. Crédito a la investigación previa de chunking y a la decisión de usar embeddings de Gemini Matryoshka (768 dim).
- Integración de `BackgroundTasks` para indexación async fue limpia. El frontend hace polling, el alumno no se entera.

Qué no salió bien

- Decidir el `min_similarity` correcto fue iterativo. 0.5 era muy estricto (descartaba chunks útiles), 0.2 muy permisivo (traía ruido). Aterrizamos en 0.3 para chat, 0.25 para buscador, 0.4 para quiz dirigido.

Acción de mejora

- Documentar thresholds en código como constantes con comentarios explicativos del trade-off.

27.1.4 Sprint 3 — Calidad y métricas

Qué salió bien

- Retries con backoff (BACK-11) cubrieron casos reales: la API de Gemini a veces devuelve 429 en horas pico, el retry los absorbe sin que el alumno note nada.
- Logging JSON estructurado permitió debugging mucho más rápido en desarrollo.

Qué no salió bien

- Implementar rate limiting demoró más de lo esperado. Decidir si era por `user_id`, por `session_id`, o global, ocupó debate más del necesario.

Acción de mejora

- Decisiones de UX como rate limiting deben pasar por un ADR corto antes de tipear código.

27.1.5 Sprint 4 — MVP completo

Qué salió bien

- Streaming SSE quedó robusto y mantuvo Hybrid PHP Proxy intacto. El HMAC sigue siendo server-to-server y el browser nunca lo ve. Latencia perceived dramáticamente mejor.

- Las **citas clickeables** (Feature D) cerraron el loop visual del RAG. Demostrable y defendible.
- Quiz generator con `response_format=json_object` evitó toda la pesadilla de parsing manual de Markdown / regex. Pydantic valida el schema.
- Detección de gaps (Feature G) con señal combinada (retrieval + respuesta del LLM) atrapa casos que ninguna de las dos detectaría sola.

Qué no salió bien

- **LLM “follow-the-example”**: el LLM copiaba literalmente el `apunte-derivadas.pdf` del ejemplo en el system prompt. No nos dimos cuenta por días hasta que un test manual lo evidenció. Fix tardío.
- Gaps iniciales solo se detectaban con chunks vacíos o similaridad baja. Fallaron los casos donde el retrieval traía chunks de alta similaridad pero **irrelevantes** (matches semánticos espurios). Agregar la regex sobre la respuesta del LLM lo arregló.
- Dos integrantes crearon migraciones Alembic con el mismo número (`004_*`) trabajando en paralelo. Linealizar el árbol fue trivial pero no debería haber pasado.
- La barra de búsqueda inicial de la pestaña Quiz aceptaba topics sin material relacionado y generaba quiz aleatorio engañoso. Fix de Sprint 4 late: devolver 404 con mensaje claro al alumno.

Acciones de mejora

- En el system prompt, **nunca usar filenames concretos como ejemplo**. Usar `[archivo.pdf]` o referencias abstractas.
- Antes de mergear migraciones Alembic, hacer `alembic heads` para detectar branches.
- Tests automatizados deben cubrir prompts de LLM con assertions de contenido (ej: “la respuesta no debe contener el string del ejemplo del prompt”).

27.2 Lecciones agregadas (retrospectiva global)

27.2.1 Lo que volveríamos a hacer

- **Investigar mucho al inicio**. Las 8 semanas de Sprint 0 ahorraron meses de refactor después. El ROI fue alto.
- **Documentar decisiones con ADRs**. Permitieron retomar discusiones zanjadas sin re-discutir y dieron material académico defendible.
- **Monolito modular sobre microservicios**. Para 2 personas y un plazo fijo, no hay otra opción razonable.
- **Multi-provider LLM desde el día uno**. Liberó al equipo de depender de un solo proveedor y permitió desarrollar el MVP gratis con Gemini.

27.2.2 Lo que cambiaríamos

- **Empezar testing automatizado más temprano**. Los tests entraron en Sprint 3 y deberían haber estado desde Sprint 1.
- **Hacer demos internas de 10 minutos por sprint**. Lo hicimos solo en Sprint 2 y 4; en Sprint 1 y 3 dimos por hecho que todos sabíamos qué se había hecho. No.
- **CI/CD desde la primera semana**. Lo deployamos en Sprint 3; debería haber sido en Sprint 0. Hubiera atrapado bugs antes.

27.2.3 Lo que aprendimos sobre IA en educación

- **Trazabilidad gana confianza.** Las pills clickeables hacen más por la credibilidad del sistema que cualquier disclaimer textual.
- **El LLM mejora cuando le decís en qué *no* tiene que confiar.** El system prompt explícito sobre admitir gaps es más efectivo que prompts optimistas.
- **El feedback loop al docente es el verdadero diferencial.** No es la IA respondiéndole al alumno — es la IA dándole al docente datos accionables sobre qué le falta a su material.

Chapter 28

Conclusiones

28.1 Logros vs objetivos

Objetivo planteado al inicio	Resultado al cierre del MVP	Estado
Plugin Moodle funcional con asistente IA	Plugin v0.9.x con 7 features deployables	☐ Cumplido
RAG auténtico sobre el material del curso	pgvector + HNSW + chunking de 512 tokens + citas trazables	☐ Cumplido
Multi-provider LLM configurable	Abstracción LLMPProvider / EmbeddingProvider con switch por .env	☐ Cumplido
Self-hosted con datos en infra de la institución	Patrón Hybrid PHP Proxy, HMAC server-to-server, sin SaaS obligatorios	☐ Cumplido
Deployable en cualquier Moodle 4.x	ZIP en GitHub Release público + instrucciones documentadas	☐ Cumplido
Streaming token-por-token	Server-Sent Events end-to-end con primer token en ~1s	☐ Cumplido
Quiz generator a partir del material	Feature F con shuffle de opciones y graceful degradation	☐ Cumplido
Detección de gaps del docente	Feature G con señal combinada (retrieval + respuesta del LLM)	☐ Cumplido
Backend deployado en producción	Railway con autodeploy desde main via GitHub Actions	☐ Cumplido
Cobertura de tests $\geq 70\%$	~80% en backend Python (37 tests automatizados)	☐ Superado

Los 10 objetivos del MVP se cumplieron. El producto entregable está en producción accesible

públicamente y el plugin se puede instalar en cualquier Moodle 4.1-4.5.

28.2 Contribución académica

NexusAI plantea una respuesta técnica concreta a tres preguntas vigentes en la IA aplicada a educación:

28.2.1 1. ¿Cómo se evita la alucinación del LLM en contextos académicos?

Con **RAG auténtico + citas trazables**. La pregunta del alumno se vectoriza, se recuperan los chunks más similares del material indexado del curso, y se inyectan al system prompt como contexto. El LLM cita el archivo fuente; el frontend muestra **pills clickable** que expanden el fragmento exacto. Si el material no cubre la pregunta, el sistema lo admite explícitamente y registra el gap para el docente — no inventa.

28.2.2 2. ¿Cómo se cierra el feedback loop alumno ↔ docente?

Con **detección automática de gaps** (Feature G). Cada vez que el sistema no puede responder bien (chunks vacíos, similaridad baja, o el LLM declara que no encontró información), persiste la pregunta en una tabla específica. El docente accede a un reporte agregado de qué temas pregunta el alumnado y que el material no cubre. Es **el sistema midiendo su propia ceguera** — contribución defendible más allá del aporte de ingeniería.

28.2.3 3. ¿Cómo se preservan la privacidad y soberanía de datos académicos?

Con el patrón **Hybrid PHP Proxy** (ADR-001) + HMAC server-to-server + backend self-hosted. La API key del LLM nunca llega al navegador. Los datos académicos viven en infraestructura controlada por la institución. La arquitectura multi-provider permite incluso usar un LLM completamente local (Ollama) si la institución tiene esa restricción.

28.3 Limitaciones del MVP

Honestidad académica sobre lo que el MVP **no** hace todavía:

- **Solo PDFs** — la pipeline soporta extracción de DOCX y TXT, pero el validador del endpoint de upload solo acepta `application/pdf` para simplificar el contrato.
- **No es multi-tenant nativo** — cada institución debe hostear su propio backend. Soportar multi-tenant con aislamiento estricto requeriría particionado por `tenant_id` en cada tabla y autenticación más sofisticada.
- **Quiz generator depende fuertemente de la calidad del LLM** — Gemini 2.5 Flash a veces genera distractores demasiado parecidos entre sí. Mejorar esto requiere prompt engineering iterativo con tribunal humano.
- **Cobertura de tests del frontend y plugin PHP es manual** — solo el backend Python tiene tests automatizados. Para producción comercial, se necesita Vitest + Behat.
- **Sin analytics agregadas para alumnos** — el docente ve gaps; el alumno no tiene dashboard de su propio uso (tokens, sesiones, temas más consultados). Planificado para post-MVP.

28.4 Trabajo futuro

28.4.1 Mejoras inmediatas (próximas 2-4 semanas post-defensa)

- Submission del plugin al **Moodle Plugin Directory oficial** (moodle.org/plugins). Aumenta la defendibilidad académica y permite que cualquier admin de Moodle del mundo instale el plugin con un click.
- Tests automatizados de frontend con Vitest + React Testing Library.
- Implementación del Privacy API completo con `metadata\provider` para cuando se agreguen tablas locales en el plugin Moodle.

28.4.2 Líneas de investigación post-tesis

- **Clustering semántico de gaps** — actualmente la Feature G agrupa preguntas por string normalizado. Con embeddings podríamos detectar que “¿qué es una matriz inversa?” y “explicame la inversa de una matriz” son la misma pregunta, y agrupar gaps a nivel temático en vez de literal.
- **Sugerencias proactivas al alumno** — basado en su historial, sugerir “te conviene repasar X antes del parcial”. Requiere modelar el progreso del alumno, no solo responder consultas.
- **Generador de planes de estudio adaptativos** — combinar gaps, quiz scores y temas no consultados para armar un plan personalizado.
- **Fine-tuning del LLM con material del curso** — alternativa a RAG cuando el material es estable y voluminoso. Permitiría respuestas más naturales sin necesidad de retrieval, pero requiere infraestructura significativa.
- **Análisis de sesgo en el material** — detectar si el material indexado cubre adecuadamente cuestiones de género, accesibilidad, perspectivas culturales diversas. Tema sensible pero relevante para tesis.

28.5 Reflexión final

NexusAI nace de una observación simple: los alumnos preguntan más a ChatGPT que al docente, y los docentes pierden visibilidad sobre qué consulta el alumnado. El sistema no compite con el docente — lo amplifica. La IA responde 24/7 con el material que el docente seleccionó, y el docente recibe un reporte agregado de qué temas no quedaron cubiertos.

Para el equipo, el proyecto fue una oportunidad de combinar varias disciplinas en un sistema real: arquitectura de software (monolito modular, patrones de auth), IA aplicada (RAG, embeddings, prompt engineering), desarrollo full-stack (PHP, Python, React, SQL+vector), y gestión de proyectos (5 sprints, 70+ issues, CI/CD, documentación de 80+ páginas).

El MVP está en producción y es entregable. El plugin se puede instalar en cualquier Moodle 4.x del mundo apuntando al backend que mantenemos en Railway. La trazabilidad del código — desde el commit que implementa cada feature hasta el ADR que justifica la decisión arquitectónica — está disponible en el repositorio público.

Queda lo difícil: que algún docente, algún curso, alguna institución, lo use. Para eso fue construido.

Chapter 29

Anexos

29.1 A. Glosario técnico

Término	Definición
ADR	Architectural Decision Record. Documento corto que registra una decisión arquitectónica con su contexto, alternativas y consecuencias.
AMD	Asynchronous Module Definition. Sistema de módulos JavaScript que usa Moodle vía RequireJS.
API key	Cadena secreta que identifica a un cliente ante un servicio. En NexusAI se usa como primera capa de auth (Bearer).
Backoff exponencial	Estrategia de retry donde cada intento espera más que el anterior (1s, 2s, 4s, ...). Usada para absorber errores transitorios del LLM.
BackgroundTask	Mecanismo de FastAPI para correr lógica async después de responder al cliente. Usado en NexusAI para indexar PDFs sin bloquear el upload.
Chunk	Fragmento de un documento procesado para embedding. Tamaño típico en NexusAI: 512 tokens con 64 de solapamiento.
CORS	Cross-Origin Resource Sharing. Política del navegador sobre llamadas cross-domain. Evitado en NexusAI por el patrón Hybrid PHP Proxy.
CSRF	Cross-Site Request Forgery. Ataque donde un sitio malicioso ejecuta acciones en nombre del usuario. Mitigado en Moodle con sesskey.

Término	Definición
Embedding	Vector denso (768 o 1.536 dim en NexusAI) que representa un texto. Textos semánticamente similares tienen vectores cercanos.
External Function	Convención de Moodle para exponer endpoints PHP a JavaScript vía core/ajax. NexusAI tiene 10 (chat, documentos, search, quiz, gaps, sessions).
FAB	Floating Action Button. El botón violeta de NexusAI que abre el chat.
HMAC	Hash-based Message Authentication Code. Firma criptográfica con secreto compartido. NexusAI usa HMAC SHA-256.
HNSW	Hierarchical Navigable Small World. Índice aproximado de pgvector para búsqueda de nearest neighbors en alta dimensión.
Hook API	Mecanismo de Moodle 4.4+ para inyectar comportamiento en eventos del core (ej: antes del footer).
Matryoshka	Tipo de embedding (como gemini-embedding-001) que permite truncar dimensiones sin perder calidad lineal. NexusAI usa 768 de 3.072 dim disponibles.
Nonce	Número usado una sola vez. NexusAI lo guarda en Redis con TTL para bloquear replay attacks.
pgvector	Extensión de PostgreSQL para columnas tipo vector con operadores de distancia.
Privacy API	Interfaz de Moodle que cada plugin debe implementar para declarar qué datos personales almacena. NexusAI usa null_provider en MVP.
RAG	Retrieval-Augmented Generation. LLM que toma contexto recuperado de una base vectorial antes de generar respuesta.
Retrieval	El primer paso de RAG: recuperar los chunks más relevantes para una pregunta.
sesskey	Token CSRF nativo de Moodle, presente en todas las requests AJAX vía core/ajax.
Similitud coseno	Métrica de distancia entre vectores normalizada al rango [-1, 1]. NexusAI la usa como $\text{score} [0, 1] = 1 - \text{distance}/2$.
Sprint	Iteración de desarrollo de duración fija (4-8 semanas en NexusAI).

Término	Definición
SSE	Server-Sent Events. Protocolo HTTP de streaming unidireccional server → client. Usado para el modo streaming del chat.
Token (en LLM)	Unidad de texto procesada por el modelo (~4 caracteres en español). El costo de la API se mide en tokens.
WBS	Work Breakdown Structure. Descomposición jerárquica de las tareas del proyecto.

29.2 B. Bibliografía y recursos consultados

29.2.1 Sobre RAG y LLMs

- Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arXiv:2005.11401.
- Karpukhin, V. et al. (2020). *Dense Passage Retrieval for Open-Domain Question Answering*.
- OpenAI Cookbook — <https://cookbook.openai.com/>
- Google AI Studio — <https://ai.google.dev/>
- Anthropic — *Building Effective Agents*, 2024.

29.2.2 Sobre Moodle

- Moodle Developer Docs — <https://moodledev.io/>
- Moodle Plugin Development — <https://moodledev.io/docs/4.5/devupdate>
- Plugin Review Criteria — <https://moodledev.io/general/community/plugincontribution>
- Hook API — <https://moodledev.io/docs/4.4/apis/core/hooks>

29.2.3 Sobre pgvector y bases vectoriales

- pgvector — <https://github.com/pgvector/pgvector>
- Pinecone — *Learning Center*, <https://www.pinecone.io/learn/>
- ANN benchmarks — <http://ann-benchmarks.com/>

29.2.4 Sobre administración de proyectos

- PMBOK Guide, 7th Edition (PMI, 2021).
- Beck, K. *Test-Driven Development by Example* (2002).
- Newman, S. *Building Microservices*, 2nd ed. (2021) — usado como referencia para decidir el monolito modular.

29.3 C. Listado completo de historias de usuario

Se incluyen en el pendrive de entrega como `NexusAI_Backlog_Completo.xlsx`. Las historias están agrupadas por épica y sprint, con criterios de aceptación y story points por cada una.

Resumen agregado:

- Total de historias: **70+**
- Cerradas al final del MVP: **70**
- Pendientes (post-MVP): **explícitamente descartadas del alcance**

29.4 D. Listado de commits relevantes

Los commits clave que implementan cada feature del Sprint 4 (con sus SHAs completos accesibles en GitHub):

Feature	SHA	Commit message
A — Buscador semántico	ef44ad6	feat: Implement multi-course messaging and semantic search features
B — Chat multi-curso	ef44ad6	(mismo commit que A)
C — Streaming SSE	6a15cd7	feat: Implement streaming chat feature with Server-Sent Events
D — Citas clickeables	27ef377	feat: Enhance message sources handling with clickable citations and previews
Fix prompt	b3d7dc6	feat: Improve system prompt instructions for citing course materials
E — Historial	9b426b9	feat: Implement chat session history features
F — Quiz generator	5f9f6c5	feat: Implement multiple-choice quiz generator
G — Detección de gaps	(incluye varios commits)	feat: teacher gap detection

29.5 E. Issues cerradas (GitHub)

Las 6 issues principales del MVP se encuentran en <https://github.com/nexusai-ucc/nexusAI/issues> con el label `mvp`. Todas están cerradas con razón “completed” y comentario que linkea al commit que las implementa:

#	Título	Estado
#253	[MVP-A] Buscador semántico	☐ closed
#254	[MVP-B] Chat multi-curso	☐ closed
#255	[MVP-C] Chat con streaming SSE	☐ closed
#256	[MVP-D] Citas clickeables con preview	☐ closed
#257	[MVP-E] Historial de conversaciones	☐ closed
#258	[MVP-F] Quiz generator	☐ closed

Además, hay 60+ issues adicionales en el repositorio que cubren backend, frontend, infraestructura y bug fixes — todas trazables vía labels `sprint:N`, `epica:*`, `prio:*`.

29.6 F. Acceso al sistema demo

29.6.1 Backend en producción (Railway)

URL del backend	<code>https://nexusai-production-e414.up.railway.app</code>
Swagger interactivo	<code>https://nexusai-production-e414.up.railway.app/docs</code>
Health check	<code>https://nexusai-production-e414.up.railway.app/health</code>
Estado	☐ Online 24/7
Mantenimiento	El equipo se compromete a mantenerlo accesible hasta la defensa final (Feb 2027)

29.6.2 Credenciales para conectar un Moodle al backend

Para que el tribunal pueda reproducir el sistema en su propio Moodle (o verificar cómo se configura), las credenciales del backend en producción son:

Campo	Valor
Backend API URL	<code>https://nexusai-production-e414.up.railway.app</code>
API key	<code>3f6db387999e0bc2b6f7ea155a983f6540a51ceea5e8a3b1</code>
Shared secret	<code>06ba2057056dc0c42814bc47b887cf2c4945156a8b9790</code>

Estos valores se pegan en **Site administration** ☐ **Plugins** ☐ **Local plugins** ☐ **NexusAI** del Moodle de destino tras instalar el ZIP del plugin.

29.6.3 Demo presencial (laptop del equipo)

Durante la defensa, el equipo levanta Moodle local en su laptop conectado al backend de Railway. Los pasos:

```
# 1. Levantar Moodle local + DB
cd ~/Documents/NexusAI/moodle-docker
./bin/moodle-docker-compose start
```

```
# 2. Verificar que el backend en Railway responde
curl https://nexusai-production-e414.up.railway.app/health
```

Una vez listo:

URL Moodle demo	<code>http://localhost:8082</code>
Usuario admin Moodle	proporcionado por el equipo durante la defensa
Usuario alumno demo	proporcionado por el equipo durante la defensa
Usuario docente demo	proporcionado por el equipo durante la defensa

El tribunal puede verificar simultáneamente desde su propio dispositivo que el backend de Railway está respondiendo: abrir `https://nexusai-production-e414.up.railway.app/health` en cualquier navegador devuelve un JSON con `status: ok`.

29.6.4 Repositorios públicos

GitHub del proyecto	<code>https://github.com/nexusai-ucc/nexusAI</code>
Release del plugin (v0.9.4)	<code>https://github.com/nexusai-ucc/nexusAI/releases/latest</code>
Issues del MVP	<code>https://github.com/nexusai-ucc/nexusAI/issues?q=label:mvp</code>
GitHub Actions (CI/CD)	<code>https://github.com/nexusai-ucc/nexusAI/actions</code>

29.7 G. Contacto del equipo

- **Santiago Tricherri** — [email institucional]
- **Delfina Salinas** — [email institucional]